DTU Compute
Department of Applied Mathematics and Computer Science

Machine Learning for Semi-Automated Annotations of the Linear A Inscriptions

Bachelor Thesis

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Richard Petersens Plads
Building 324
2800 Kongens Lyngby, Denmark
www.compute.dtu.dk

Preface

This Bachelor's thesis was prepared at the Department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a Bachelor of Science in Engineering (BSc Eng) degree in General Engineering (Cyber Systems).

Kongens Lyngby, June 7, 2026

Frederik W. L. Christoffersen

Abstract

To enable computational analysis of the still undeciphered Linear A script of Bronze Age Crete, epigraphers such as Dr. Ester Salgarella have been working to structurally digitize its corpus. Since this process is slow and laborious, involving manual drawing and labeling, this thesis investigated whether the drawing step can be partially automated by approaching the problem as a segmentation task on raw documents, using the Segment Anything Model (SAM). Due to the small corpus size and lack of labeled data, the study tested whether zero-shot and few-shot variants of SAM can transfer to the epigraphic domain of Linear A without training or fine-tuning. Three aspects were evaluated: non-interactive segmentation quality, tool efficiency, and the total time saved in the full workflow. Across all tests, SAM did not outperform the strongest classical baseline. In a real-world case study, the tool even seemed to increase total work time, as its output required too much correction. These results suggest a general need for a sufficiently sized registered dataset of epigraphic documents, as SAM otherwise lacks the domain-specific knowledge to perform well on this task. For Ester's workflow specifically, they further suggest that the drawing step may need to be approached differently than as a task of segmentation on raw document images. Data and code are available at: `https://github.com/FrederikWLC/linA-scribe`.

Acknowledgements

Frederik W. L. Christoffersen
General Engineering (Cyber Systems), DTU
Author of this thesis

Morten Mørup
Professor, DTU Compute
Main supervisor

Ester Salgarella
Research Fellow, AIAS Aarhus / University of Cambridge
Co-supervisor, creator of SigLA, and winner of the 2026 Michael Ventris Award in Mycenaean Studies

Lukas Esterle
Associate Professor, Aarhus University
Co-supervisor

Generative AI tools were occasionally used as support for brainstorming, linguistic editing, code development, and research during the development of this project.

Thanks to Morten for making this project possible. Thanks to Lukas for his tips and support in developing the fundamental idea for it. And many thanks to Ester for her positive attitude, the resources she provided, and the time she put into it. I am grateful for the opportunity to have worked with you.

Contents

Acronyms

AIAS Aarhus Institute of Advanced Studies

API Application Programming Interface

CannyFill Canny edge detection with morphological closing

Cefael Collections de l'École française d'Athènes en ligne

CNN Convolutional Neural Network

CV Computer Vision

Dice Dice-Sørensen coefficient

FATESAM Few-shot Adaptation of Training frEe SAM

GAN Generative Adversarial Network

Gaussian Gaussian adaptive thresholding

GFSAM Graph-based Few-shot Segment Anything Semantically

GORILA Godart and Olivier, Recueil des inscriptions en linéaire A

HT Haghia Triada document

ICL In-Context Learning

IPR Intellectual Property Rights

IoU Intersection over Union

KH Khania document

MA Mallia document

ML Machine Learning

MLP Multilayer Perceptron

MVP Minimum Viable Product

NLP Natural Language Processing

Otsu Otsu's method

POI Point of Interest

RQ Research Question

SA-1B Segment Anything 1 Billion

SA-V Segment Anything Video

SAM Segment Anything Model

SAM+pts SAM with automatic point prompts

SOTA State of the Art

TPE Tree-structured Parzen Estimator

ViT Vision Transformer

YOLO You Only Look Once

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Background

When the British archaeologist Arthur Evans in the beginning of the 20th century went to excavate Knossos on Crete, he found a huge amount of clay tablets written in a previously unknown linear script. Some documents were clearly of a newer script, which he called *Linear B*, while others were of an older one, which he named *Linear A*. Some fifty years later, the British amateur archaeologist Michael Ventris finally succeeded in deciphering the newer script Linear B, which he proved to be representing Mycenaean Greek, the earliest attested form of Greek. But the same hypothesis did not hold for the older script Linear A, whose encoded language remained unknown, still yet to be deciphered.

A key obstacle to its decipherment has been the small size of its corpus, lately consisting of only around 2,500 inscriptions, and about 1,400 if only including those well-preserved enough to be readable (Salgarella 2025, p. 13). This is significantly smaller than that of its younger counterpart, which now spans more than 4,500 inscriptions that are also comparatively longer and better preserved (Salgarella 2025, p. 46). But new inscriptions are still being discovered, with the corpus most recently extended in 2025 by over 100 new documents (Del Freo and Zurbach 2025; Consiglio Nazionale delle Ricerche 2025)

Another challenge has been the lack of digital representations of these inscriptions, in a form suitable for statistical analysis. While photographs accompanied by expert drawings of nearly the entire corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA) - are available online through Collections de l'École française d'Athènes en ligne (Cefael), these are only available in print form (Godart and Olivier 1976-1985). This motivated the development of the palæographi-

cal[1] database *The Signs of Linear A* (SigLA), which seeks to "[develop] a systematic, exhaustive and user-friendly open access database of all Linear A inscriptions [...] in order to carry out statistical and palæographic analyses within the epigraphic[2] corpus" (Salgarella and Castellan 2020). This database has already been utilized as a source for computational analysis and Natural Language Processing (NLP) of Linear A (Chen et al. 2026). However, until a most recent addition of four documents in May 2026 and a few changes to its source code since April 2026, the latest expansion of SigLA had been in 2021 (Salgarella and Castellan 2026a; Castellan 2026). And the project has so far managed to document just 776 of all currently known inscriptions (Salgarella and Castellan 2026b).

1.1.1 A problem of manual digitization

The drawings currently stored on SigLA were made by Ester Salgarella using the painting tool Krita. For each document, she manually created her own annotated drawings based on the GORILA originals, stored by Cefael as poor quality gray-scale raw tablet images accompanied by expert drawings. From these, she proceeded to create layered Krita files with metadata for each sign, which could then at last be imported as JSON and Krita files to SigLA (Salgarella and Castellan 2020). According to her (personal communication), this has been a highly time-consuming undertaking - and the necessity of this kind of labor-intensive work has arguably been the biggest reason to why SigLA still does not yet contain the entire available corpus.

The aim of this project was therefore to investigate whether parts of this process can be automated through processing of the tablet images from GORILA. Furthermore, due to the potential violation of Intellectual Property Rights (IPR), it was of particular interest, whether the annotation process could begin from the raw source images themselves, without relying on the undigitized but already completed expert drawings.

[1]The term *palæographical* refers to the study of historical writing systems.
[2]The term *epigraphic* refers to the study of inscriptions.

1.1.2 A problem of small data

The small size of the Linear A corpus, and the irregularities in quality of its raw documents, is a huge problem for the application of deep learning, as models created therefrom typically require large amounts of high-quality data to perform well.

For more informative and better quality data, there does exist an online repository of 3D scans of Aegean inscriptions[3]. That is, through the project INSCRIBE (Ravanelli, Lastilla, and Ferrara 2022), which via photogrammetry[4] sought to "produce the first complete digital corpus of all three Aegean undeciphered scripts". But this project ended in 2023, and has so far just 71 Linear A documents scanned (ERC INSCRIBE Project 2026).

Consequently, this thesis instead sought to make use of generalized Machine Learning (ML) and Computer Vision (CV) through inference from the GORILA corpus of 2D images. In particular, it aimed to utilize Meta's State of the Art (SOTA) interactive segmentation model, the Segment Anything Model (SAM) (Kirillov et al. 2023), which is pre-trained on a massive and diverse dataset - Segment Anything 1 Billion (SA-1B), of over 1 billion masks. SAM's generalization and interactivity (taking user-placed prompts that guide the segmentation) potentially allows for semi-automatic annotation of Linear A inscriptions without the need for large amounts of training data, which was unfortunately not available in this context. Therefore, it was of particular interest to map the capabilities and limitations of SAM within this domain.

1.2 Related work

Within the last decade, the field of document annotation has seen significant advancements through the use of machine learning. Take for example the EU-funded generic CNN-based framework for historical document processing *dhSegment* (Oliveira, Seguin, and Kaplan 2018). Or the more

[3]The term *Aegean inscription* refers to the ancient writings found near the Aegean sea, including the scripts Linear A, Linear B, and Cretan Hieroglyphics.

[4]*Photogrammetry* is a technique used to create 3D models from overlapping 2D images.

recent work on segmentation of stone inscriptions (P. Zhang, C. Li, and Sun 2024), where SAM and other state-of-the-art methods are beaten as benchmarks by their stacked U-Net and GAN architecture. Or even the YOLO and Roboflow based approach to segmentation of Palmyrene Aramaic inscriptions (Hamplová et al. 2024) that augment[5] their dataset with a Generative Adversarial Network (GAN) to increase performance.

Nevertheless, these methods all depend on deep learning, making them inconvenient for domains of small data, which instead seem to require a generalized and interactive approach such as SAM.

1.2.1 SAM and its applications

Ever since its release in 2023, SAM has been extensively evaluated across a wide range of CV tasks to investigate whether it can really "segment anything". Papers have evaluated its performance within niche domains, such as medical images (Ma et al. January 2024), camouflaged objects (Tang, Xiao, and B. Li 2023), transparent objects (Han et al. 2023), hierarchical texts (Ye et al. 2024), and even oracle bone inscriptions (Wang et al. 2025) and Maya hieroglyphs (Shivam et al. 2024). Most of these papers found that SAM still tends to struggle with such domains, despite its impressive zero-shot[6] generalization. Meanwhile, some of them also found that this performance can be significantly improved through architectural extensions (Wang et al. 2025) and domain-specific fine-tuning (Ma et al. January 2024; Shivam et al. 2024). While such fine-tuning does not always require very large datasets, it does require task-specific annotated data and model training, which was outside the scope of this thesis. which unfortunately requires deep learning and a lot of data. However, equally, it has been shown that SAM's performance can significantly improve through effective prompt engineering alone (Price, Rundensteiner, and Cote 2025), requiring no deep learning at all. And within the domain of 3D medical images, a training-free few-shot[7] adaptation of SAM2 (see Ravi et al. 2024) has been recently demonstrated to surpass supervised methods like U-Net, and even the best fined-tuned versions of SAM

[5]The term *augmenting* refers to the process of increasing the size of a dataset.
[6]The term *zeroshot* refers to model inference/prediction on hitherto unseen data.
[7]The term *few-shot* refers to model inference/prediction with only a few examples.

(He et al. 2025b). Thus, it seemed reasonable to test out the limits of SAM's generalization and interactivity within the domain of Linear A document images, to investigate whether it could be used to accelerate the annotation workflow of Ester Salgarella.

1.3 Problem statement

As previously stated, the aim of this project was to investigate whether the drawing step of the process of creating annotated drawings like the ones in SigLA can be automated by working on the raw tablet images from GORILA. More specifically, whether some kind of semi-automatic segmentation tool (given just the raw tablet images as input) using interactive and generalized CV - specifically the Segment Anything Model (SAM) - can accelerate the workflow of the drawing step, while maintaining high-quality outputs that are still compatible with a database like SigLA. But what would be the relative segmentation quality of such a tool? How interaction efficient would it be exactly? And by how much could it potentially improve the efficiency of the workflow? To concretize the research, the problem statement was decomposed into three Research Questions (RQs) - each accompanied by their respective operationalization (see the following subsection for details).

1.3.1 Research questions

1. **RQ1 (Segmentation quality)**: *How well does a SAM-based segmentation approach perform quantitatively in terms of segmentation accuracy, compared to simple classical image processing baselines when applied to Linear A document images?*

   Operationalization: SAM-based masks were compared to classical image-processing baselines using overlap metrics (i.e. Dice) against the ground truth segmentations derived from SigLA's Krita files using a self-made dataset of 60 documents.

2. **RQ2 (Interaction efficiency)**: *To what extent can user interaction be reduced by a SAM-based semi-automatic segmentation*

workflow, while still maintaining high-quality outputs?

Operationalization: Two self-developed segmentation tools - one SAM-based, one classical - were evaluated in a case study of 3 documents. The time spent in each tool by Ester Salgarella from image import to mask export was compared against the segmentation quality (i.e. Dice) relative to ground truth (SigLA's Krita files).

3. **RQ3 (Workflow efficiency)**: *How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions?*

   Operationalization: The time it took Ester Salgarella to annotate manually compared to being assisted by two self-developed segmentation tools - one based on SAM, and the other based on a classical baseline - was measured in a case study of 3 documents.

1.4 Impact – innovation and application

Concerning its digitization, Linear A already has printed corpus material via GORILA (Godart and Olivier 1976-1985), a palæographical database via SigLA (Salgarella and Castellan 2020) including roughly half of the corpus, and a repository of high-quality 3D scans via INSCRIBE (Ravanelli, Lastilla, and Ferrara 2022) including less than five percent of the corpus. But these address only the problem of access - not the efficiency of the digitization process itself. To the best of the author's knowledge, no prior published or publicly documented study has investigated whether Linear A inscriptions can be automatically extracted via segmentation from the widely available GORILA raw tablet images.

Broadly speaking, SAM and related segmentation models have already been tested out for similar purposes, including extraction of Mayan hieroglyphs (Shivam et al. 2024). Studies also show that SAM can be generally adapted to niche domains without either tuning or training (He et al. 2025b), but leave open whether it transfers to the Linear A tablet

imagery characterized by constrained conditions of small data and low image quality.

Although Ester Salgarella already has a well-established annotation workflow, she does not yet have a tool that can give her a useful first draft of the drawing to save her time. No prior testing exists of whether a generated mask from the GORILA raw tablet images can serve as such. Nor, to the best of the author's knowledge, has any prior attempt been made to automate either the drawing step or the metadata-labeling step of this annotation process.

The work of this thesis thus presents itself as a first proof-of-concept tool and evaluation framework for testing whether SAM and segmentation in general can assist Linear A annotation in practice. Once such a tool proves effective, it could potentially also be applied to Linear B or other epigraphic scripts, as it is the whole field of Aegean studies (according to Ester) that seems to suffer from severe under-digitization of the primary evidence (which is a problem for the scientific investigations of scholars). Hopefully, the work will inspire further research and development within the field of digital epigraphy, particularly concerning automation of the annotation processes of Linear A - and perhaps even in the broader context of small corpus epigraphic writing systems, where the lack of big and high-quality data heavily restricts deep-learning approaches.

1.5 Overview of the thesis

The rest of this thesis is structured into four main chapters (Data, Methodology, Results and Discussion, and Conclusion), all split individually into sections concerning each of the three research questions.

CHAPTER 2

Data

2.1 Sources

The dataset used in this thesis was constructed from 60 images of raw Linear A inscriptions, and their corresponding ground truth annotations. Cefael (Godart and Olivier 1976-1985) was chosen as the source for raw document images, since it is the only online and widely publicly available source of the printed five volume corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA). As ground truth, annotations provided by Ester Salgarella from SigLA (Salgarella and Castellan 2020) were chosen instead of the annotated drawings from the corpus, as the former are more up to date, and of better quality.

What follows in this chapter is a detailed description of this dataset, documenting its characteristics, its construction process, and its limitations and biases.

2.2 Characteristics and construction

The dataset was made to consists of 60 chosen image pairs of raw documents and their corresponding ground truth annotations. Each of these image pairs was categorized into one of three difficulty levels (easy, medium, hard) based on the visual quality of the inscriptions (see Section 2.2.1). They were then randomly split into three subsets: test (27 images), validation (27 images), and support (6 images), with each one having an equal proportion of images from each of the three difficulty levels. All reported results of this Thesis are based on the test set, while the validation set was used for hyperparameter optimization, and the support set was used for ICL variants of SAM (see Sections 3.1.3.3 and 3.1.3.4).

Only inscriptions with clay as writing support were included, as they are the most common, representing over 90% of the Linear A corpus (Salgarella 2025, p. 29), and were the only ones for which SigLA ground truth annotations were already available. The raw document images from Cefael are screenshots of grayscale images placed within the scanned printed edition of the corpus, and are thus not of the highest quality, being the only widely publicly available images of the Linear A inscriptions. For an example of a raw document image and its corresponding ground truth annotation, see Figure 2.1.

((a)) Raw image (HT7b) **((b))** Ground truth (HT7b)

Figure 2.1: Example of a raw inscription image and its corresponding ground truth annotation.

2.2.1 Difficulty classification

The dataset was categorized into three difficulty levels (easy, medium, hard) based on the visual quality of the raw images and the clarity of

the inscriptions. This classification was performed manually by inspecting each image and considering factors such as contrast, noise, and the presence of tablet breakages or general damage to the inscriptions. The concrete criteria used to classify the difficulty sets were as follows:

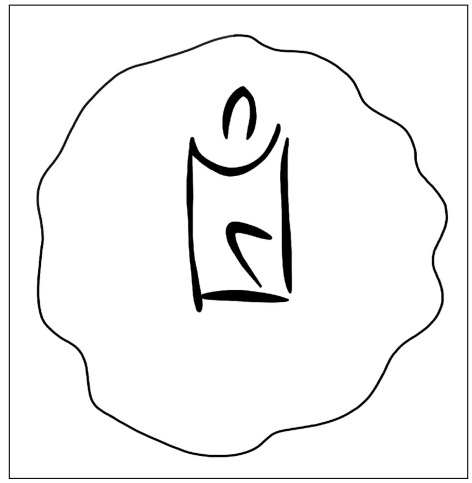
- **Easy:** labeled as such for documents with clear ingravings, good contrast, minimal noise, and constant lighting.
- **Hard:** labeled to those with poor contrast, significant noise, and substantial damage making it hard even for the naked eye to read the inscriptions.
- **Medium:** labeled to all those inbetween the qualities of easy and hard.

For visual examples of images within each of these difficulties, see Figure 2.2.

2.2.2 Image registration and preprocessing

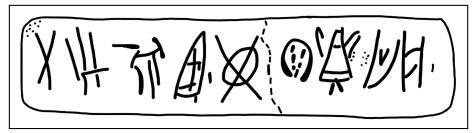
To obtain any kind of meaningful comparison between model outputs and ground truth, the raw images and their ground truth annotations had to be aligned and registered to each other. This was done manually in Krita for all 60 image pairs by downscaling the ground truth images to the individual scales of their raw counterparts, and aligning by translation and rotation, using the raw images as background reference.

((a)) Easy KHWc2104 (raw)

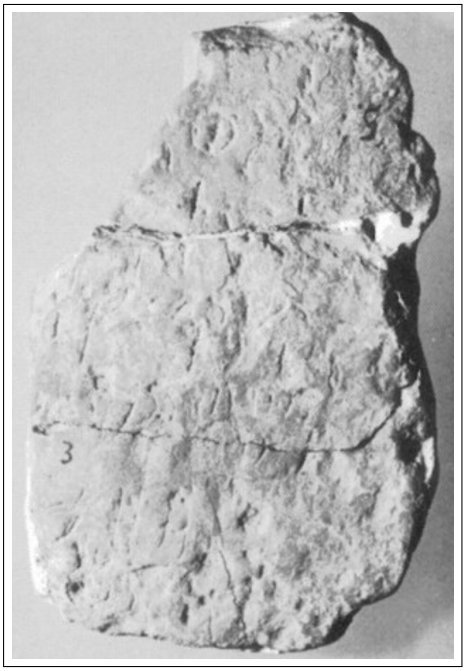


((b)) Easy KHWc2104 (gt)

((c)) Medium MA1a (raw)



((d)) Medium MA1a (gt)



((e)) Hard HT3 (raw)

((f)) Hard HT3 (ground truth)

Figure 2.2: Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.

2.3 Limitations and biases

2.3.1 Quality of the raw tablet images

Due to the weak quality of the raw tablet images, they carry only parts of the visual information of the tablets they represent. When the drawings of GORILA were made, the researchers had access to the tablets themselves, meaning they could see what the images did not pick up, and better discern actual engravings from mere damages to the tablets. Therefore, these tablet images are far from the best source of data there is, being however, the only source publicly available. That lack of good quality data can unfortunately limits the output quality of segmentations constructed therefrom.

2.3.2 Alignment to the ground truth

Even after image registration, ground truth images still did not perfectly align with the raw images, affecting the obtained segmentation quality. This misalignment was a source of noise in the evaluation. Ester's annotations were never constructed by overlaying them on the raw images, but rather by doing so on the accompanied corpus drawings, only using the raw images as reference.

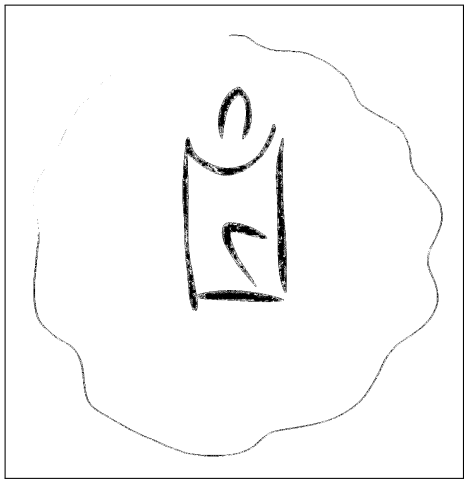
As seen in Figure 2.2(f), the drawing style of the drawings is also especially biased with dots and and lines representing damages on the tablets, which are not present in the same way visually in the raw images. Thus, the raw images and the ground truth annotations were never perfectly aligned to begin with, and the manual registration process could only do so much to mitigate that. This is seen by the low Dice values obtained for all models in the results (see Section 4.1).

2.3.3 Binarization of the drawings

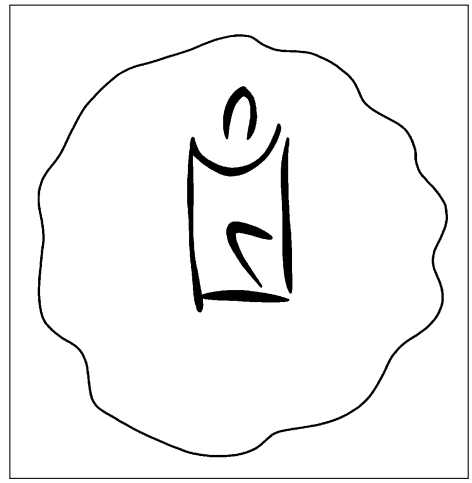
Due to a missing threshold step of the registered JPEGs in the initial registration of the dataset, the ground truth annotations were to begin with binarized with a threshold of 0 instead of 128, meaning that only

the pure black pixels of the drawings were considered as part of ground truth foreground, while the smoothed gray pixels were not accounted for. This led to lower Dice scores in general, as the ground truth then in practice was thinner and more sparse compared to the actual drawings.

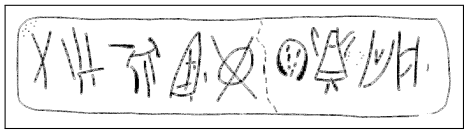
This issue was for the evaluation later corrected to a threshold of 128, meaning that all pixels with higher intensities now were considered foreground, and all pixels with lower intensities were considered background (see Figure 2.3 for visual examples of the two different ways of binarization). However, only the hyperparameter optimization of the models was not redone for the new binarization format, and was thus performed with ground truth in the old binarization format. Arguably, this only affected the effectiveness of the optimization and what the models were optimized for (penalizing non-conservative segmentation styles more than otherwise), while still preserving fairness and generalizability, as all model hyperparameters were tuned under the same conditions.



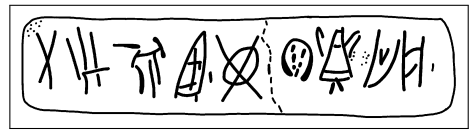
((a)) KHWc2104 (old)



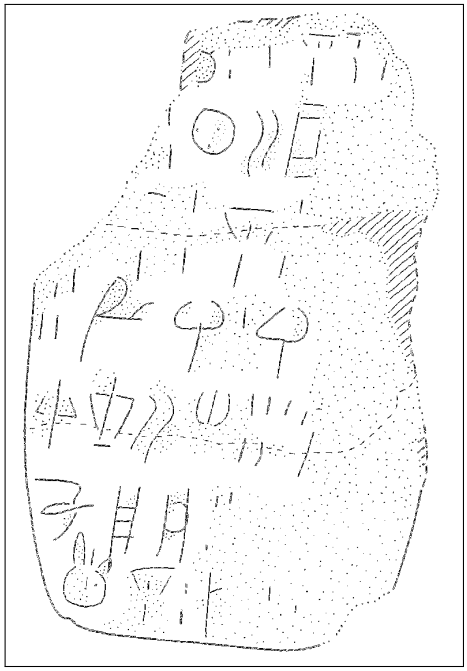
((b)) KHWc2104 (new)



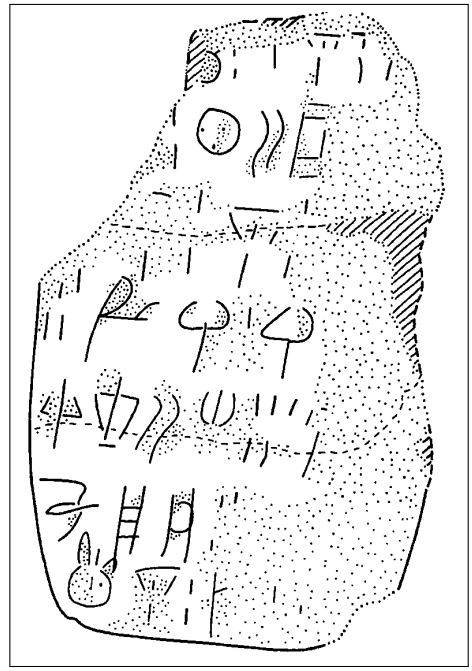
((c)) MA1a (old)



((d)) MA1a (new)



((e)) HT3 (old)



((f)) HT3 (new)

Figure 2.3: Examples of images processed with the two different binarization thresholds; 0 (old) and 128 (new).

CHAPTER 3
Methodology

This chapter describes in detail the operationalization done to answer the question of whether a SAM-based segmentation tool can accelerate the annotation workflow of Linear A documents (see Section 1.3.1 for an overview of the individual RQs). Overall, this operationalization was achieved through a comprehensive evaluation of segmentation quality (see Section 3.1), and a case study involving Dr. Ester Salgarella measuring the interaction efficiency (see Section 3.2) and workflow efficiency (see Section 3.3) of a self-developed SAM-based segmentation tool. In that way, these three abstract virtues were decomposed into independent measurable metrics, making it possible to concretely answer the research questions, and where the potential benefits and drawbacks of SAM lie.

3.0.1 Development environment and tools

All software written and data collected for this thesis was made publicly available on GitHub (Christoffersen 2026). Python 3.13 was used for all code development except frontend. The choice of this programming language was made primarily due to it being the language of choice for the ML and SAM implementation libraries needed. Its big community, and library support, and package manager (pip) also made it exceptionally easy to install and manage dependencies, when seeting up the development environment. The particular version was chosen, as it was the second latest stable release at the time of development, with the risk of Python 3.14 still not having full support by all libraries.

3.1 Non-interactive segmentation

To answer the question of RQ1: *How does the initial segmentation quality of SAM compare to that of classic segmentation methods?*, the initial fully-automatic segmentation quality of SAM was evaluated against a set of classical baseline models, using a suitable segmentation metric (i.e. Dice score) computed from the pairs of predicted and ground truth masks. All models were tuned through hyperparameter optimization (see Section 3.1.4.2) to their best performance on the dataset, and the best performing version of SAM was compared against the best performing baseline model.

The following subsections give an explanation of the segmentation problem itself, and document the configuration of the baseline and state-of-the-art models, as well as the evaluation protocol used to answer RQ1.

3.1.1 The problem in mathematical terms

The task of extracting the foreground (engraving and outline) from the background of an image of a writing support can be conceptualized as a binary image segmentation and classification problem at the pixel level, where each pixel is classified as either belonging to the foreground or the background. Although this segmentation is binary, it is single-class in the sense that all foreground pixels are treated as a single foreground class, without distinguishing between different types of foreground, like tablet outlines, sign types, etc. This segmentation process can be mathematically described as a mapping that, given some predicate function f, and an input image I, produces a binary mask B of the same dimensions, where each pixel value indicates whether it belongs to the foreground (by convention denoted as 1) or the background (denoted as 0):

$$B(x, y) = \begin{cases} 1 & \text{if } f(I, x, y) \\ 0 & \text{otherwise} \end{cases}$$

In a threshold-based approach for example, the predicate function can be expressed as $f(I, x, y) = I(x, y) > T$, where T is the threshold value that determines the cutoff between foreground and background pixels (Gonzalez and Woods 2018, p. 743). The predicate function does not,

however, have to depend only on the individual pixel value being classified. It can for example also be defined based on an attribute computed from the global image context (see global thresholding with Otsu in Section 3.1.2.2). Or it can be determined from the local context of each pixel's neighborhood (see local adaptive thresholding in Section 3.1.2.3), which can be expressed by kernel convolution working on neighborhood intensities of a window $w \in \mathbb{R}^{m \times n}$ around each pixel (Gonzalez and Woods 2018, p. 157, 159) (where $a = (m-1)/2$ and $b = (n-1)/2$ are the vertical and horizontal half-widths of w, respectively, assuming odd dimensions):

$$(w * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} w(i, j) I(x - i, y - j)$$

Even the entire image including external guidance can be given as context, as in the case of SAM (see Section 3.1.3.1), where the inner workings of its transformer-based structure can be understood as a learned, non-linear black-box function that takes both image and prompts to produce the binary output mask. Thus, the segmentation problem can evidently be approached by many different methods, each with their own contextual arguments and inner workings f. And of those evaluated, it was just a matter of choosing the best one (for the evaluation details used to do that, see Section 3.1.4).

3.1.2 Baseline model configuration

The classical baseline models against which SAM was evaluated (from simplest to most advanced) were:

- Global thresholding (Otsu's method)
- Local adaptive thresholding (Gaussian)
- Edge-based segmentation (Canny edge detection with region filling)

Together, they cover a range of classic segmentation techniques that might be used for binary foreground extraction. Besides a description of the pre-filtering method used for all of them, what follows is a brief

description of each of these methods, including their characteristics, how they work, and their implementation details.

3.1.2.1 Bilateral pre-filtering

As a pre-filtering step for all the classical models, a bilateral filter (see Tomasi and Manduchi 1998) was applied to the input image before feeding it to the classical methods. In this way, more noise could be reduced while preserving engravings, as this filter is known to be effective at blurring out noise while preserving edges, thanks to its combined distance and intensity-dependent weighting. Discretely (see Pan 2025), its output value $I_{\mathrm{bf}}(p)$ at a given two-dimensional pixel position p (where q is one of its neighbors within the kernel window with size d) can be defined as follows:

$$I_{\mathrm{bf}}(p) = \frac{\sum_q I(q)\, W_{\mathrm{space}}(p,q)\, W_{\mathrm{color}}(p,q)}{\sum_q W_{\mathrm{space}}(p,q)\, W_{\mathrm{color}}(p,q)}$$

Here, the spatial weight $W_{\mathrm{space}}(p,q)$ based on the distance between pixels p and q, and the color range weight $W_{\mathrm{color}}(p,q)$ based on the intensity difference between p and q are respectively given by:

$$W_{\mathrm{space}}(p,q) = \exp\left(-\frac{\|p-q\|^2}{2\sigma_{\mathrm{space}}^2}\right)$$

$$W_{\mathrm{color}}(p,q) = \exp\left(-\frac{\|I(p)-I(q)\|^2}{2\sigma_{\mathrm{color}}^2}\right)$$

The method thus takes three hyperparameters: the kernel window diameter d, the spatial standard deviation σ_{space}, and the intensity standard deviation σ_{color}. However, inspired by (OpenCV 2026b), it was decided to use the same σ value for both spatial and color weights for the implementation, as this simplified the hyperparameter tuning (see Section 3.1.4.2) without significantly compromising the filter's performance.

3.1.2.2 Global thresholding (Otsu's method)

Otsu's method is a global thresholding method that works on gray-level histograms, which was originally proposed by Otsu (1979). As the simplest of the baselines, it represents the minimum acceptable segmentation

quality, since it only takes the global gray level distribution into account, with no assumptions about any spatial relationships between pixels. It is therefore expected to perform poorly on images with erosion, uneven lighting, and differences in surface texture (Gonzalez and Woods 2018, p. 751), which are the exact characteristics of the images to be segmented.

Here is how it works: First, let pixels of the raw image be divided into L shades of gray $[1, 2, ..., L]$. Then, let each pixel be assigned one of two classes C_0 and C_1, guided by a threshold T, where C_0 contains the shades $[1, 2, ..., T]$, and C_1 contains the rest $[T + 1, T + 2, ..., L]$. Otsu's method then works by performing an exhaustive search over all L gray levels to find an optimal threshold T^* that results in a maximum between-class variance σ_B^2:

$$T^* = \underset{1 \leq T < L}{\arg\max}\ \sigma_B^2(T)$$

The between-class variance σ_B^2 is then defined by a function of the respective total probabilities of the two classes (ω_0 and ω_1) and the mean gray level value per class (μ_0 and μ_1):

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_1(T) - \mu_0(T)]^2$$

Without the bilateral pre-filter, this method has no hyperparameters.

3.1.2.3 Local adaptive thresholding (Gaussian)

Local adaptive thresholding works by computing a threshold for each pixel based on its local neighborhood. This is an advantage in cases where the image has varying light conditions (which is the case for the images to be segmented), as the threshold adapts to local lighting.

Mathematically, the local threshold T is a function of a given pixel position (x, y) computed by a weighted sum of the neighborhood. The generated binary mask B is then defined as (Gonzalez and Woods 2018, p. 761):

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T(x, y) \\ 0 & \text{otherwise} \end{cases}$$

To compute $T(x, y)$, there are a few weighting functions to choose from. For this baseline, the Gaussian kernel was used, as it can limit the influence of distant noisy pixels on the computed threshold by assigning weights based on distance from the kernel center. With a Gaussian kernel G_σ, the threshold $T(x, y)$ is computed as a Gaussian-weighted sum of local intensities subtracted by a constant C. Using kernel convolution $(*)$, this can be expressed as (see OpenCV 2026a, p. 761):

$$T(x, y) = (G_\sigma * I)(x, y) - C$$

In general, the weight in the Gaussian kernel at an offset (i, j) from the kernel center can be written as follows (where α is a normalization constant ensuring that the whole kernel sums to 1, and σ controls the spatial spread of the kernel) (Gonzalez and Woods 2018, p. 167):

$$G_\sigma(i, j) = \alpha \cdot \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right)$$

Thus, without the bilateral pre-filter, this method has two hyperparameters: the kernel size d, and the constant C.

3.1.2.4 Edge-based segmentation (Canny edge detection with region filling)

Canny edge detection is a method that by itself only segments the edges of an image, excluding the enclosed regions within those edges. First, for the image gradient of each pixel, the magnitude $M_s(x, y)$ and the orientation $\theta_s(x, y)$ are computed (Gonzalez and Woods 2018, p. 730):

$$M_s(x, y) = \| \nabla I_s(x, y) \| = \sqrt{\left(\frac{\partial I_s}{\partial x}\right)^2 + \left(\frac{\partial I_s}{\partial y}\right)^2}$$

$$\theta_s(x, y) = \arctan\left(\frac{\partial I_s / \partial y}{\partial I_s / \partial x}\right)$$

Next, *non-maxima suppression* (Gonzalez and Woods 2018, p. 730-731) is applied to retain only local maxima of $M_s(x, y)$ along the gradient

direction $\theta_s(x, y)$, resulting in thinner edges. This is then followed by *hysteresis thresholding* (Gonzalez and Woods 2018, p. 732), classifying pixels as strong, weak, or non-edges with a low threshold T_l and a high threshold T_h, only preserving weak edges if connected to strong edges.

Inspired by (Rosebrock 2015), for the implementation, these thresholds were automatically determined by introducing a hyperparameter σ that controls the sensitivity of the edge detection (where v is the median of the grayscale values of the given image):

$$T_l = \max\Big(0, (1-\sigma)\, v\Big), \quad T_h = \min\Big(255, (1+\sigma)\, v\Big)$$

Then, in an attempt to fill the regions enclosed by the detected edges, morphological closing (dilation followed by erosion) was applied to the edge mask.

Hence, without the bilateral pre-filter, this method has two hyperparameters: the constant σ, and the kernel size d used for morphological closing.

3.1.3 State-of-the-art

To be compared against the classical baselines was the generalized deep learning Segment Anything Model (SAM) (Kirillov et al. 2023). Due to its popularity as a SOTA segmentation model, and the fact that its different use cases require different trade-offs in terms of accuracy, speed, and domain adaptation, there are many different variants of SAM. For the sake of fairness, it seemed only reasonable to test out the following range of them:[1]

- The original SAM (see Kirillov et al. 2023), and its computationally efficient variant MobileSAM (see Zhao et al. 2023)
- The newer SAM2 (see Ravi et al. 2024)
- The few-shot variant FATESAM (see He et al. 2025b) and one-shot variant GFSAM (see A. Zhang et al. 2024a)

[1] Due to access restrictions of the checkpoints (weight files) necessary for running the newly released SAM3 (Carion et al. 2025), it was not included in the evaluation.

Together, these variants span the axes of relevance, recency and ICL capabilities that should make for a fair test.

Furthermore, for additional fairness, a small ablation study was conducted on the base models (SAM, MobileSAM, SAM2) to investigate whether these variants benefitted from the use of bilateral pre-filtering (see Section 3.1.2.1), and automatic threshold-derived point prompts (see Section 3.1.3.5).

To be able to run all these computationally demanding variants of SAM, model execution was outsourced to Modal (see Modal Labs 2026) – a serverless cloud computing platform usually used for ML inference, tuning and training. The following subsections describe the architecture of the chosen variants, and how they were implemented in the evaluation.

3.1.3.1 SAM architecture

Used by SAM and MobileSAM, the first SAM model architecture released (Kirillov et al. 2023) consists of an image encoder, and a comparatively lightweight prompt-guided mask decoder (see Figure 3.1). This image encoder takes the form of a Vision Transformer (ViT), that similarly to a Convolutional Neural Network (CNN) processes an input image in order to extract its features (Dosovitskiy et al. 2021). Instead of relying on just convolutional layers, however, a ViT adapts the transformer architecture originally developed for NLP to the image domain. Rather than words, an image is treated as a sequence of fixed-size patches (regions of the image), each of which is embedded into a vector representation and augmented with a positional encoding, analogous to how tokens are handled in NLP. Seen in Figure 3.2, these embedded patches are then passed through a repeated layered series of normalization, learned feed-forward Multilayer Perceptrons (MLPs), and, in between, a learned mechanism known as self-attention that weighs the embeddings according to their relevance to one another.

More specifically (according to Vaswani et al. 2023), given an input embedding X, self-attention computes three projections; the queries Q, keys K, and values V, as products of X with their respective learned matrices, and passes the information on as a new matrix (where d_k is the

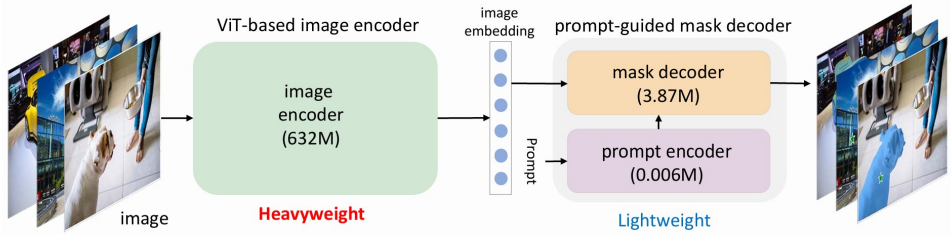


Figure 3.1: Architecture of SAM, from C. Zhang et al. (2023). An input image is processed by a ViT-based image encoder to produce an image embedding, which is then fed to a comparatively lightweight (also transformer-based) prompt-guided mask decoder along with embedded prompts to generate the final segmentation mask.

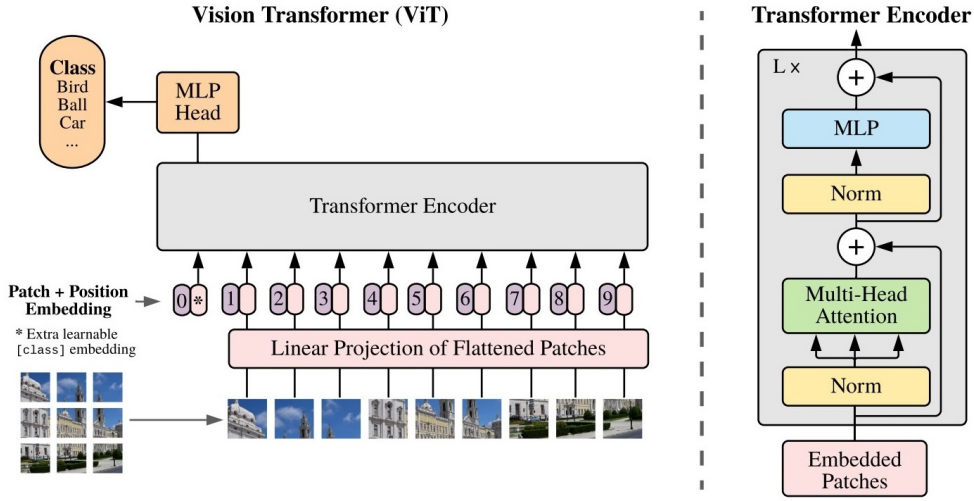


Figure 3.2: Architecture of the ViT, from Dosovitskiy et al. (2021). An image is treated as a sequence of flattened fixed-size patches, each embedded into a vector representation augmented with a positional encoding. These embeddings are then passed through a repeated layered series of normalization, learned feed-forward Multilayer Perceptrons (MLPs), and learned self-attention.

dimension of Q and K):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here, the softmax normalization function turns the scaled query–key similarities $\frac{QK^T}{\sqrt{d_k}}$ into a set of attention weights (each on the interval $]0, 1[$, in total summing to 1), which determine how the value vectors V are combined into the input of the next layer.

Finally, the ViT outputs an image embedding that captures the spatial context of the image. It is this image embedding, combined with embedded prompts from the prompt encoder (referred to as prompt tokens), that is fed to the mask decoder in order to generate the final segmentation mask. Before this can happen, however, the prompt encoder must convert any given sparse prompts (i.e. box or point prompts) into embeddings, which is done by summing their positional encodings with learned type-specific embeddings (one foreground/background embedding per point prompt, and two opposite-corner embeddings per box prompt).

Seen in Figure 3.3, these are then fed to the lightweight mask decoder (which is also transformer-based) along with the image-embedding from the ViT, and three learnable output token embeddings (see Figure 3.3). However, for a generation of single output masks, which is what was used in this thesis for all models except GFSAM, the mask is generated from just a single output token (Kirillov et al. 2023, p. 17),

Self-attention is then applied to the prompt and output tokens, and two-way cross-attention (same as self-attention but with one embedding as queries, and the other as keys and values, and vice-versa) is applied between the tokens and image embedding for them to influence and attend one another. To keep track of spatial coordinates and prompts, the image embedding, and the original prompt embeddings along with their positional encodings are added as input at every attention layer.

At the end of this process, the image embedding is upsampled by convolution to the original image resolution. The output tokens are then processed in parallel by two types of MLPs: one that generates the final segmentation masks by taking a dot product between each token and the upscaled image embedding, and another that outputs a predicted IoU score for each mask.

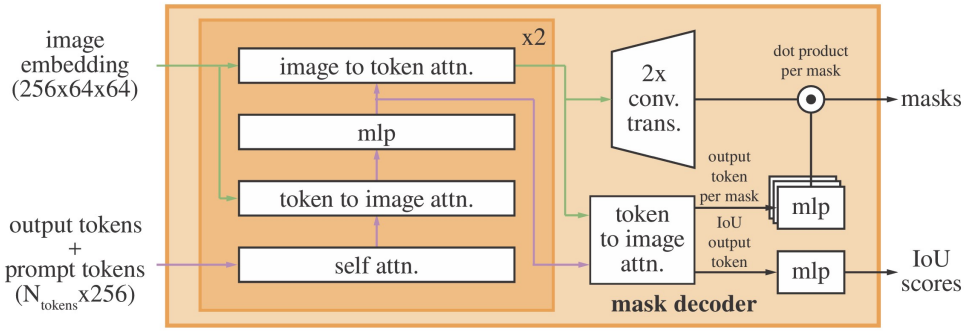


Figure 3.3: Architecture of SAM's mask decoder, from Kirillov et al. (2023). The image embedding, embedded prompts, and learnable output tokens interact through self-attention and two-way cross-attention, after which the image embedding is upsampled by convolution and the output tokens are processed by two types of MLPs: one whose output is combined with the upsampled image embedding via a dot product to generate the segmentation masks, and another that outputs a predicted IoU score per mask.

For the implementation of SAM in the evaluation, the ViT-H checkpoint was used, as it is the largest and best-performing of the available variants. MobileSAM, by definition, instead relied on the lighter ViT variant TinyViT (Wu et al. 2022), whose 5M parameters are far fewer than those of the SAM encoders (ViT-H, ViT-L, and ViT-B, with 632M, 307M, and 86M parameters, respectively). It is this reduction in encoder size that makes MobileSAM computationally faster than the original.

3.1.3.2 SAM2

With the release of SAM2 (Ravi et al. 2024), the image encoder was changed to Hiera (Ryali et al. 2023) – a hierarchical ViT, which by lowering attention complexity is more parameter efficient than vanilla ViTs, as tokens are pooled together at different stages of the encoding process. Additionally, a memory attention mechanism was added for consistency, allowing the model to attend to segmentation outputs of previous frames to make masks between frames consistent enough to segment videos. As

seen in Figure 3.4, it does so by storing a memory-encoded version of the mask output of each previous frame in a memory bank that are then attended to by the image-encoded embeddings before being passed to the mask decoder.

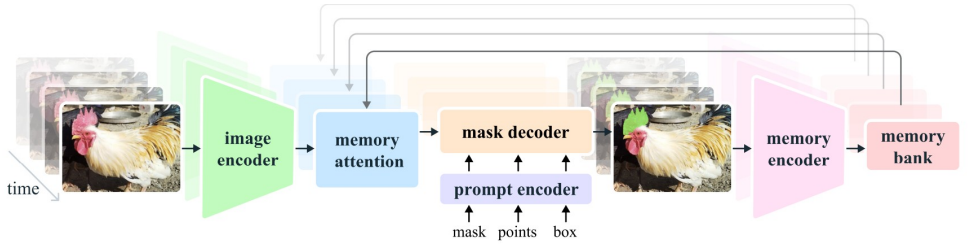


Figure 3.4: Architecture of SAM2, from Ravi et al. (2024). A learned memory attention mechanism was added, allowing the model to segment videos. For each frame, the output mask is encoded by a learned memory encoder and stored in a memory bank, whose contents are attended to by the image-encoded embeddings before being passed to the mask decoder.

SAM2 was pre-trained on both still images and video, first on the SA-1B image dataset (Kirillov et al. 2023) and then on the Segment Anything Video (SA-V) dataset (Ravi et al. 2024). The mask decoder did not change much from the original (Figure 3.3), except for the passing of early visual features from the Hiera image encoder to the convolutional upsampling process, improving the integrity of the output mask. But also added were object pointers as lightweight vectors keeping track of the semantic information of each object of interest. These are also stored in the memory bank together with the spatial memory, and attended to in memory attention.

As for the implementation of SAM2 in the evaluation, the checkpoint chosen was `sam2.1_hiera_large`, as it was the best performing one listed in the repository of SAM2 (Meta AI Research 2024). SAM2 also takes a checkpoint-specific YAML configuration file, which stores the hyperparameters for the model, and allows for easy overrides through Hydra. Since the tablets were segmented as individual still images, SAM2's image predictor was used rather than the video predictor. The memory and video-tracking machinery is only relevant to the FATESAM adaptation described in the next section.

3.1.3.3 FATESAM

Within the domain of 3D medical images, FATESAM – a training-free few-shot adaptation of SAM2 (see He et al. 2025b) has been recently demonstrated to surpass even the best fine-tuned versions of SAM. As seen in Figure 3.5, it works in principle by making use of SAM2's architecture and memory attention mechanism, by passing a few support images to its memory encoder, such that the mask decoder ends up with context to generate a similar segmentation mask for the input query image.

At inference of the 2D-adaptation made, support masks y_s^j (rather than support mask volumes y_s^{ij} with index slice i) are chosen by the similarity of their corresponding image-encoded support images $\mathcal{E}(x_s^j)$ to the image-encoded query image $\mathcal{E}(x)$ to make sure that they are as representative as possible. Like the original, this ranking is computed ascendingly from the Manhattan distances between the image-encoded features of the input and the support images. As memory encoded embeddings $\mathcal{ME}(y_s^j)$, these support masks are then passed to memory attention $\mathcal{MA}$ together with the image-encoded query image $\mathcal{E}(x)$, whereafter the output is passed on to the mask decoder $\mathcal{D}$ to generate the final mask y.

The adaptation of FATESAM in the evaluation was based on the official repository (He et al. 2025a), and the checkpoint used was `sam2_hiera_large`, as it was the best performing one used by the repository. Three support images were injected per query image, as this was the number of support units that was found to be most optimal in the paper (He et al. 2025b). To further adapt FATESAM to single frame images, the object pointer functionality was turned off through Hydra/YAML overrides, which was necessary to avoid shape mismatches. In particular, the object-pointer encoder and object-score prediction heads were disabled by setting `use_obj_ptrs_in_encoder`, `pred_obj_scores`, `pred_obj_scores_mlp`, and `fixed_no_obj_ptr` all to false. Attempts were made to contact the authors of FATESAM about the correctness of this adaptation, but no response was received. Interestingly, in the ablation study of object pointers in SAM2 (see Ravi et al. 2024, p. 15), its importance was only evaluated on video datasets and included on that basis. It was hence deemed reasonable to assume that it would not contribute to the performance of FATESAM on single frame images, and thus could be turned off without compromising performance.

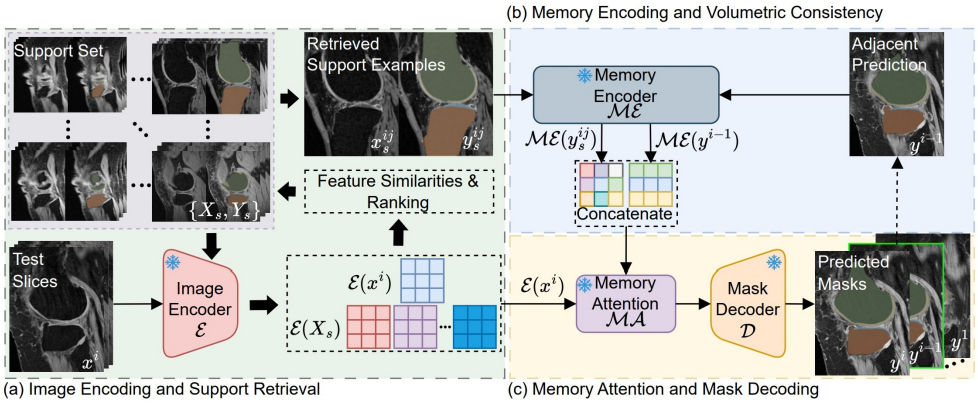


Figure 3.5: Architecture of FATESAM, from He et al. (2025b). At inference of the 2D-adaptation, support masks y_s^j are first chosen (a) by the similarity of their corresponding image-encoded support images $\mathcal{E}(x_s^j)$ to the image-encoded query image $\mathcal{E}(x)$ and then passed (b,c) to memory attention $\mathcal{MA}$ as memory encoded embeddings $\mathcal{ME}(y_s^j)$. Unlike the original's 3D inference, no adjacent predictions y^{i-1} are relevant, since a Linear A image is treated as an independent 2D input frame x.

3.1.3.4 GFSAM

The other kind of training-free ICL adaptation of SAM tested was the one-shot GFSAM (see A. Zhang et al. 2024a). As seen in Figure 3.6, it works by first extracting the features of the given support image and the query image by using the pre-trained image encoder DINOv2 (Oquab et al. 2024).

With its Positive-Negative Alignment module, GFSAM then computes two similarity maps between query and support features: one for positive/object similarity, and one for negative/background similarity. These maps are used to generate foreground point prompts for SAM, which are then fed to the mask decoder that generates one mask per point prompt. A graph is then constructed of the coverage of the points lying within each of the generated masks, whereafter the removal of some of the points representing weakly connected components is determined through processes referred to as Positive Gating and Overshooting Gating. The purpose of those gating processes is to filter out false-positive masks, where the former process makes sure a given point is actually

foreground according to the similarity maps, and the latter ensures that the point is not a boundary-near outlier. Finally, the masks associated with the remaining points are merged together to generate the final mask. If instead no points remain after the gating processes, the mask with the highest predicted IoU score is selected as the final mask.

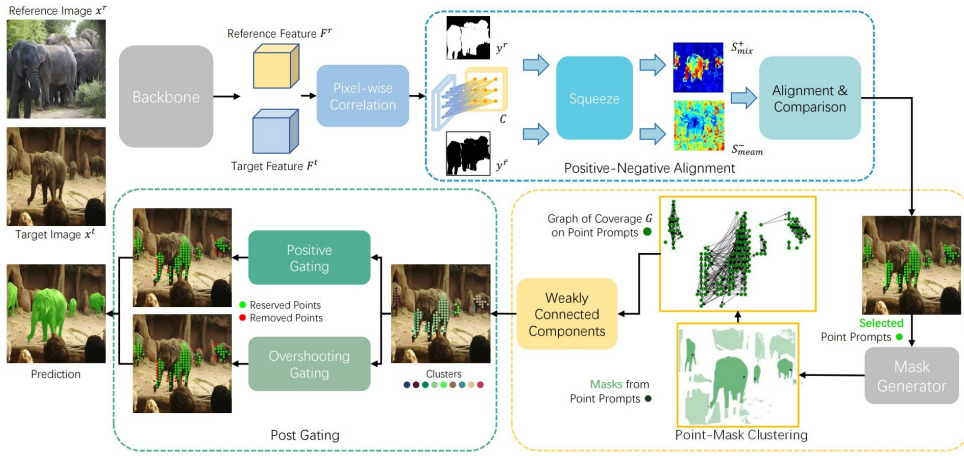


Figure 3.6: Architecture of GFSAM, from A. Zhang et al. (2024a). Features of the one-shot support and query images are first extracted with the pre-trained DINOv2 encoder, from which the Positive-Negative Alignment module computes object- and background-similarity maps. These maps give rise to foreground point prompts for SAM, which generates one mask per point. A graph of the points' coverage within each of these masks then removes weakly connected, false-positive points via Positive and Overshooting Gating, whereafter the masks of the surviving points are merged into the final segmentation mask.

The implementation of GFSAM in the evaluation was based on the official repository (A. Zhang et al. 2024b) with its DINOv2 feature encoder using the ViT-L/14 checkpoint, and SAM using the ViT-H checkpoint. As an adaptation to the data domain, the one-shot support image was selected in the same way as for FATESAM: the support images were ranked by the Manhattan distance of their DINOv2 features to those of the query image, and the most similar one was selected as support.

3.1.3.5 Threshold-inferred automatic point prompts

Another way to adapt SAM to the specific characteristics of the domain was through the use of automatic prompts. As it is particularly difficult to automatically create meaningful box prompts without a trained object detection model, only point prompts were used for this adaptation.
Inspired by Point of Interest (POI) placement in (Price, Rundensteiner, and Cote 2025), automatic point prompts for SAM were derived from the output mask of the Gaussian thresholding baseline (see Section 3.1.2.3). This was done by randomly sampling a number of foreground and background pixels from the classical output mask as prompts for SAM. Concretely, n_{fgd} foreground points and n_{bgd} background points were sampled from the foreground and the eroded background of the mask, respectively (see Figure 3.7 for a visual example). Without the hyperparameters of Gaussian, this method thus introduces three hyperparameters: n_{fgd}, n_{bgd}, and the erosion kernel size d.

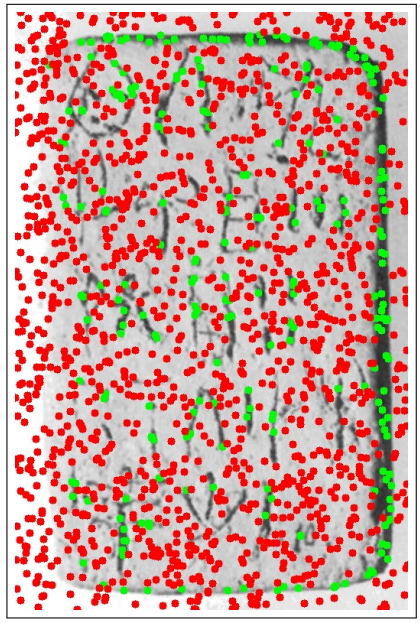


Figure 3.7: A visual example of prompt distribution for the POI prompt strategy, overlaid on HT7a. Green points represent foreground point prompts, while red points represent background point prompts.

3.1.4 Evaluation protocol for RQ1

To determine whether one model performed significantly better than another, a relevant metric (Dice) and a statistical test (paired t-test) needed to be defined. Additionally, to ensure a fair comparison, all models with hyperparameters were optimized to their best performance on the validation set. What follows is a description of this evaluation protocol.

3.1.4.1 Choice of metric (Dice)

When it comes to evaluation of segmentation models, the two most commonly used metrics are the Dice-Sørensen coefficient (Dice) and the Intersection over Union (IoU). The Dice score is most commonly used in cases where there are heavy class imbalances (e.g. lots of background compared to regions segmented), and when foreground objects are small and thin. It is naturally more forgiving than the related segmentation metric IoU, which is more strict and specifically used to evaluate segmentation results where small displacements are critical for results. Therefore, segmentation performance was reported using Dice. Mathematically, it is defined as twice the size of the intersection of the predicted segmentation mask $\hat{Y}$ and the ground truth mask Y, divided by their summed size (Mohammadi, Mollazade, and Behroozi-Khazaei 2025):

$$\text{Dice}(\hat{Y}, Y) = \frac{2 \cdot |\hat{Y} \cap Y|}{|\hat{Y}| + |Y|}$$

The raw metrics thus ended up consisting of Dice scores produced per image for each model.

3.1.4.2 Hyperparameter optimization (TPE)

All models with hyperparameters were tuned to their best Dice scores on the validation set in a series of 100 trials each using Optuna, a hyperparameter optimization framework. The optimization itself was performed using the Tree-structured Parzen Estimator (TPE) algorithm (Bergstra, Yamins, and Cox 2013; Bergstra, Bardenet, and Kégl December 2011), which is a Bayesian optimization method that models the relationship

between hyperparameters and performance to efficiently explore the hyperparameter space and find the optimal ones. Hence, it is more sophisticated than a random search, which just samples different hyperparameter combinations without considering the results of past evaluations.

It works by building two probabilistic models: one for the configurations that have resulted in good performance, and another for those that have resulted in poor performance. It then uses these models to select the next hyperparameter configuration to evaluate, selecting the one that maximizes the expected improvement. This way, it can focus the search on promising regions of the space of possible hyperparameters, removing the need to rely on chance or exhaustive search.

3.1.4.3 Statistical test (paired t-test)

To determine whether the observed difference in Dice scores between the two strongest models was statistically significant, a paired t-test was conducted. Specifically, a one-tailed paired t-test was used, as the alternative hypothesis was that SAM would perform significantly better than the best classical baseline, instead of just significantly differently. The paired t-test has two main assumptions: 1. independence of tested observations, and 2. normality of the paired differences. The first assumption is already satisfied a priori, since the segmentation performance of one image does not directly influence the performance on another. However, the second assumption depends on the results. Thus, the paired differences in Dice scores between the two models tested needed to be checked for normality using a QQ-plot and the Shapiro-Wilk test. In case the normality assumption was violated, the non-parametric domain-specific Wilcoxon signed-rank test would be used instead (Rainio, Teuho, and Klén 2024).

3.2 Tool-side interaction

To answer the question of RQ2: *To what extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs?*, a two-in-one interactive segmen-

tation tool was developed including a method meant to represent SAM, and another to represent the classical segmentation methods, to allow for a direct comparison in terms of tool efficiency. The following subsections go into details on how this tool was developed, the experimental design for the evaluation of interaction efficiency, and the definition of interaction metrics.

3.2.1 Tool development

The tool was developed in close collaboration with Ester Salgarella, in an attempt to make it fit her workflow and needs as much as possible. However, before any user tests were conducted, a Minimum Viable Product (MVP) was developed based on the workflow that she had described throughout the fortnightly supervisor meetings attended during the writing of this Thesis from February 2nd 2026, and the few sporadic meetings and written exchanges since November 2025 that took place during the planning of the project proposal. When this MVP was ready, three usability tests were conducted with Ester, two of them online, and the last one in-person at Aarhus Institute of Advanced Studies (AIAS).

During these usability tests, Ester was asked to perform the segmentation of at least one document with each method of the tool, while thinking aloud and giving feedback on the user experience. Small modifications were then made to the tool based on her feedback, until the final acceptance test was conducted. These tests took place the 5th, 8th and 12th of May (see Section A.1.1 for documentation).

3.2.2 Tool architecture

The architecture of the tool (see Figure 3.8) was developed as a web-application using Svelte as the frontend framework, and FastAPI as the backend framework, hosted via Render (Render 2026). Because it was made a web application, it could be run remotely on any device through a web browser, without the need for installation, making it more easily accessible and ideal for online user testing with Ester Salgarella. To make the development environment more consistent with production, the application was containerized using Docker, which made it easier to de-

ploy and run on any machine without worrying about dependencies and environment setup.

Svelte was chosen for the frontend mostly due to its reactive state management, making it well-suited for building interactive user interfaces, as this functionality removes the need for re-inserting state variables every time they change. FastAPI was chosen as the backend, as it is Python based, and therefore able to serve the Python segmentation models implemented, which was essential for the tool's functionality. It is also lightweight and asynchronous, which potentially allowed for many users to use the tool simultaneously, as several requests could be handled at the same time without blocking execution on the server.

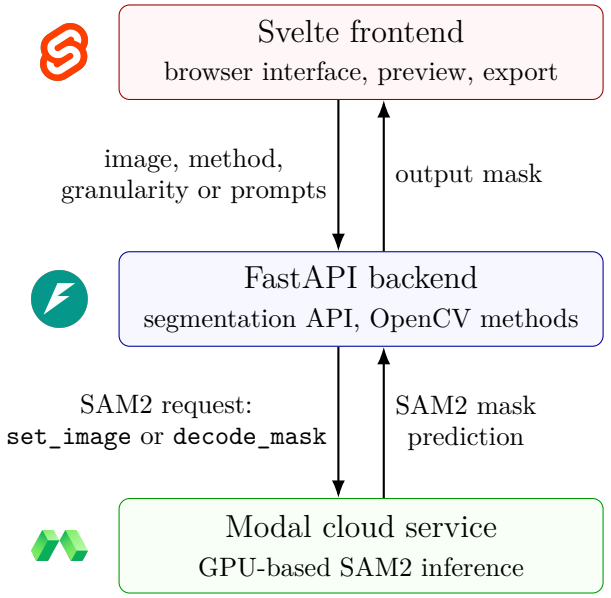


Figure 3.8: Software architecture of the developed segmentation tool, showing the communication between the browser frontend, backend API, and cloud-based SAM2 inference service.

As previously mentioned, the tool was split into two segmentation methods: one method meant to represent SAM, and another meant to represent the classical segmentation methods. Besides the general user interface, the following subsections go into more details on how these were implemented.

3.2.2.1 General user interface

The user interface of the tool was designed to be as simple and intuitive as possible, while providing the necessary functionality for the user to perform the segmentation task. The main view of the tool consisted of a canvas where the user could see the image imported (through paste from the clipboard, file upload or drag-and-drop). Once an image was imported, it was displayed on the canvas, and the user could press the "Run" button to generate a mask with the chosen segmentation method on the image. Thus, the user could then toggle between two views: one to see the original image imported; "Raw", and another to see the segmented mask generated therefrom; "Mask" (see Figure 3.9 for an example of this view, using the classical method). And at last, the user could then export this mask in a format of their choice (see Section 3.3.2 for details on this).

3.2.2.2 Integration of the classical method

Chosen to represent classical segmentation methods was the best classical method from the non-interactive evaluation, Gaussian adaptive thresholding (see Sections 3.1 and 4.1.3). On the basis of user feedback during the usability tests (see Section A.1.1), a slider was added as an interactive way to control the coarseness of the segmentation. A range of "granularity" was given $[-50, 100]$, where 0 represented the default hyperparameters of the model. The higher the value of this "granularity" slider, the more coarse the segmented mask would turn out, and the lower the value, the more fine-grained it would be. In that way, the user could adjust the coarseness of the output mask without having to worry about the specific hyperparameters of the model, and without having to understand how they work. This worked by defining the Gaussian kernel size D_{Gaussian} as a function of the granularity value as follows (where g is the value of granularity, and $D_{\text{Gaussian,default}}$ denotes the default kernel size of the model):

$$D_{\text{Gaussian}} = \max\left(5,\ D_{\text{Gaussian,default}} + 2\left\lfloor\frac{g}{5}\right\rfloor\right)$$

The granularity value was thus divided by 5 (making the intuitive

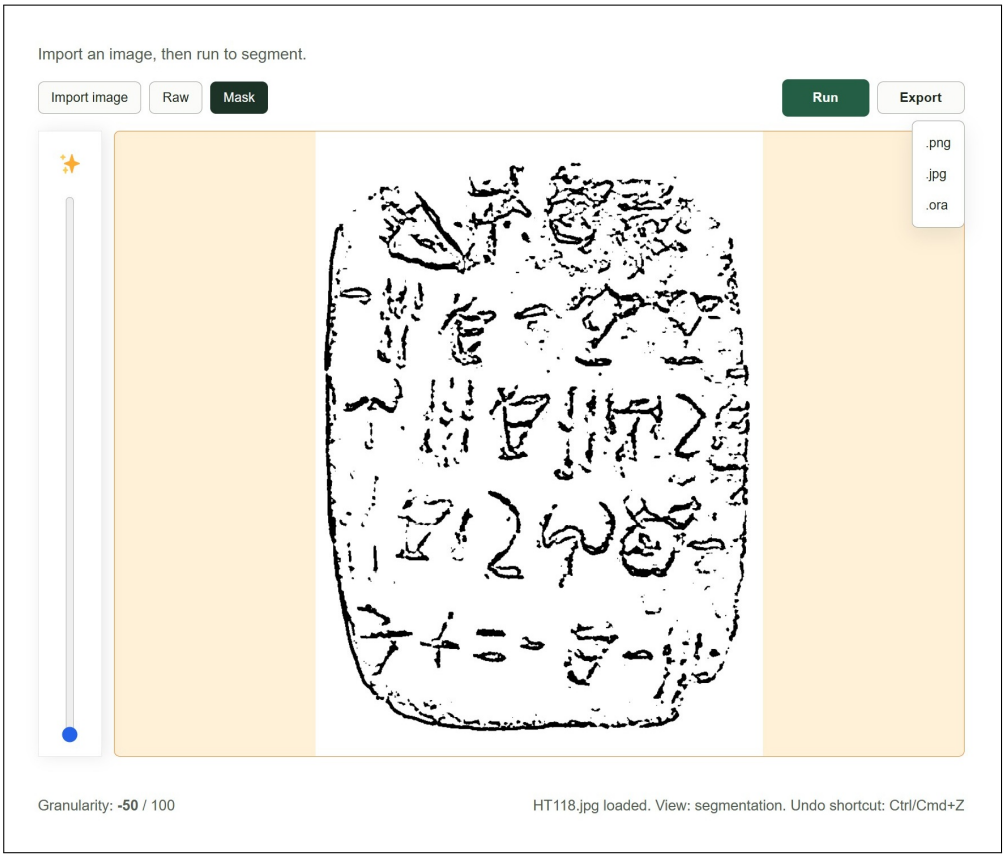


Figure 3.9: In-tool view of the classical Gaussian adaptive thresholding tool in the mask view.

larger percentage-style range $[-50, 100]$ possible), and then rounded down to the nearest integer, to determine how many times the kernel size should be increased by 2 (to keep it odd), with a minimum kernel size of 5 to avoid meaningless too fine-grained outputs. A view of this method and the granularity slider can be seen in Figure 3.9, where the slider is located in the toolbar on the left.

3.2.2.3 Integration of SAM

Chosen to represent SAM was the vanilla version of SAM2. This choice introduced an internal-external selection asymmetry, as the classical method was selected based on the preceding empirical evaluation (Section 4.1.3), whereas vanilla SAM2 was selected on external empirical grounds due to its documented SOTA performance in general interactive segmentation (Ravi et al. 2024). This was deemed appropriate because the non-interactive evaluation described in the previous section could not determine how SAM would perform when used interactively.

It was decided to make full use of its interactive capabilities, including both box prompts, and foreground and background point prompts, as this seemed to be the best way to represent its potential. But there was one problem. The mask decoder for SAM2 (and SAM for that matter) only takes one box prompt at a time, which is not ideal when wanting to segment multiple signs of interest within the same tablet. To circumvent this, the prompts given to it when boxes were present were processed in the mask decoder so that a separate mask was generated once for each n box prompts placed, also passing only the point prompts within it (see Figure 3.10 for the associated user interface). These n masks were then at last merged together by a pixel-wise logical OR operation to generate the final segmentation mask.

This was still sensible computationally, as predictions were split up such that the image encoder was only executed once on image import, while the mask decoder (as described in Section 3.1.3.1) is lightweight enough that it could be executed multiple times without a significant increase in latency. And even though one strongly oversegmented mask from any single box could inflate the combined result, each predicted mask was expected to remain largely confined to the prompted region due to the nature of box prompts.

Included in the prompt placement interface of the tool was a toolbar consisting of the following tool buttons (see Figure 3.10 and Figure 3.11 for in-tool views of the SAM-based tool, with the toolbar on the left):

- `Box prompt`: When selected, allowed the user to place box prompts on the canvas by clicking the pixel coordinates of the corners that should define the box.

- `Foreground prompt`: When selected, allowed the user to place foreground point prompts on the canvas by clicking on the pixel coordinates they want to include in the segmentation.
- `Background prompt`: When selected, allowed the user to place background point prompts on the canvas by clicking on the pixel coordinates they want to exclude from the segmentation.
- `Eraser`: When selected, allowed the user to remove any placed prompts by clicking on them.
- `Undo`: When selected, allowed the user to undo the last prompt-related action performed.
- `Reset`: When clicked, cleared all prompts from the canvas, allowing the user to start over.

3.2.3 Experimental design for RQ2

To evaluate the interaction efficiency of the SAM-based tool, a case study was conducted with Ester Salgarella, where she was asked to perform the segmentation of 3 documents from the test set (one from each difficulty category) with each method of the tool. In a wish to obtain statistical power, an inclusion of all 27 test images was initially preferred, but due to time constraints, only 3 documents made it to the evaluation. That reduced sample size made the evaluation exploratory and case-study-based rather than statistically powered. However, it still allowed for a direct comparison of the interaction efficiency of the two tools, using quantitative indicators of both time expenditure (minutes), and the quality of the outputs they generated (Dice). The interaction burden could then further be exploratorily decomposed into the time taken for the models to execute, and for SAM-based tool, the amount and types of prompts placed.

The experiment itself was conducted in-person with Ester Salgarella as the participant at AIAS on the 12th of May on the same day as the last usability and acceptance test. Specifically, she was instructed to make use of the tool method evaluated on the document given until

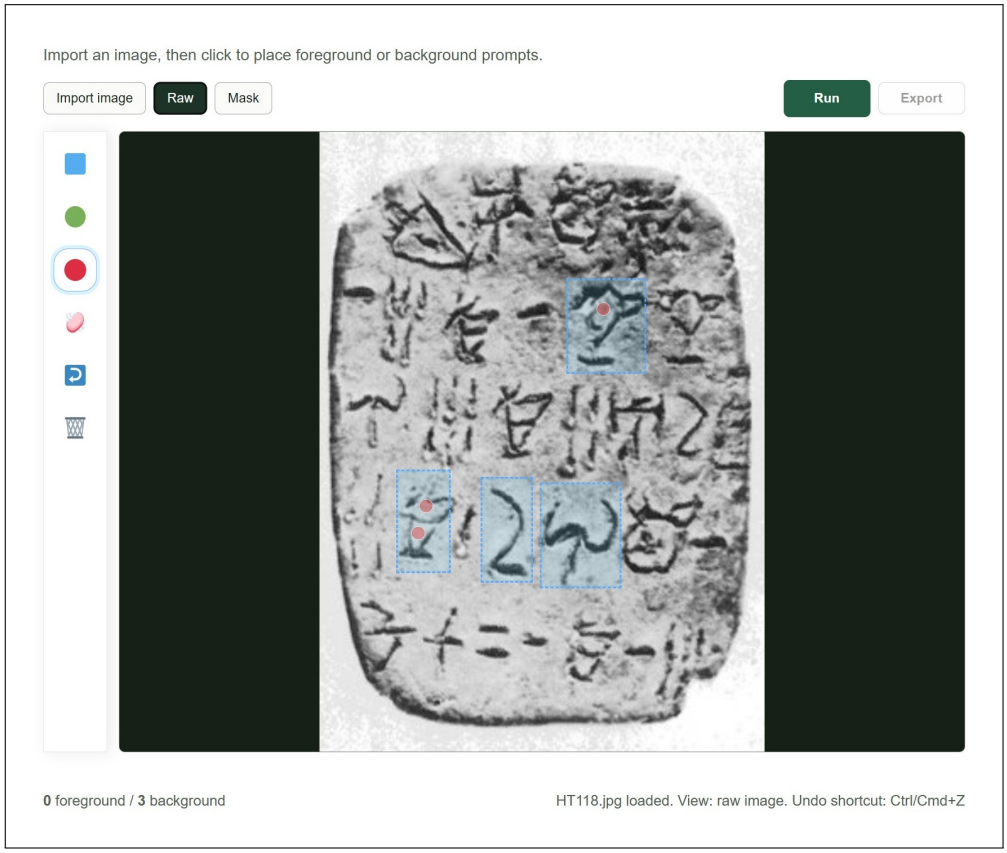


Figure 3.10: In-tool view of the SAM-based tool in the raw view, with box prompts (in blue) placed around some signs of interest, with correcting background point prompts (in red) placed within the oversegmented regions.

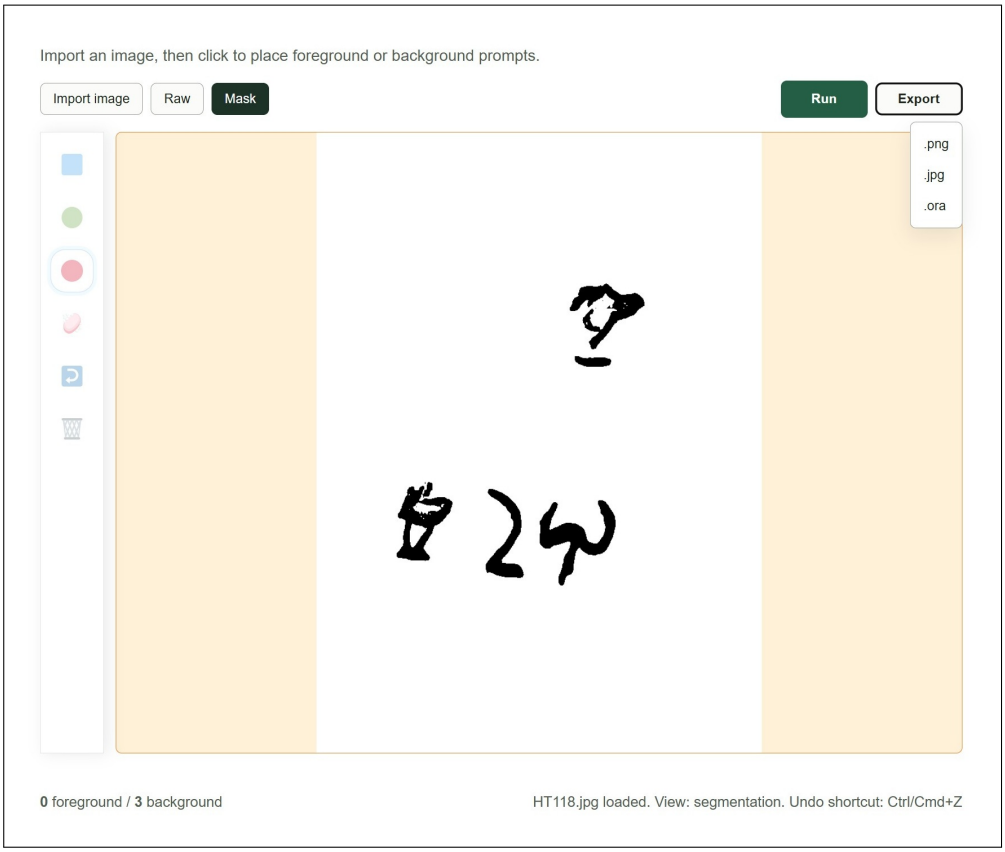


Figure 3.11: In-tool view of the SAM-based tool in the mask view, before export of the mask generated from the prompts placed as seen in Figure 3.10.

she deemed that she could not further improve the segmentation output, whereafter she should export the final mask as it was. The order of the methods tested was not randomized: the classical tool was tested first, followed by the SAM-based tool. Possible order effects, such as fatigue, are therefore treated as a methodological constraint. Tool interaction was recorded by the help of live screen recordings via Microsoft Teams, wherefrom the model execution time, the prompts placed, and the time taken from image import to mask export were all measured. The exports of each tool were also saved on disk, so that Dice scores could later be computed therefrom.

3.3 Workflow integration

To answer the question of RQ3: *How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions?*, another case study of 3 documents was conducted with Ester Salgarella, where the time taken to finish polishing the segmentation outputs exported from the two developed tools (see Section 3.2) was measured, and compared to the time taken to do the same drawings fully manually.

The following subsections go into more details on the workflow definition, the attempt made to fit the tool into it, and the experimental design for the evaluation of this workflow efficiency.

3.3.1 Definition of the workflow

The tool was designed to fit into Ester's existing workflow as much as possible, to make it more likely that she would be able to use it with minimal friction, and to make the evaluation of its efficiency more ecologically valid. To do that, however, it was necessary to first understand her workflow in detail. Figure 3.12 illustrates this workflow and where the tool was designed to fit into it, up to the marked *scope cutoff*.

When Ester does the annotation of a document from the GORILA corpus, she first imports a screenshot of the GORILA corpus drawing into Krita, creating a new layer on top of it to do the segmentation drawing. She draws this drawing with a digital pen on a displayless pressure-sensitive Wacom drawing pad. While doing that, she looks for any possible mistakes in the signs, with the original image, and her mental map of the sign types as guidance, correcting them by erasing and redrawing. At last, when she is satisfied with the drawing, she splits each sign into a separate layer, adds metadata, and imports to SigLA as JSON and Krita files.

Due to scope limitations, it was only the process before that splitting into layers and adding of metadata that this project aimed to improve.

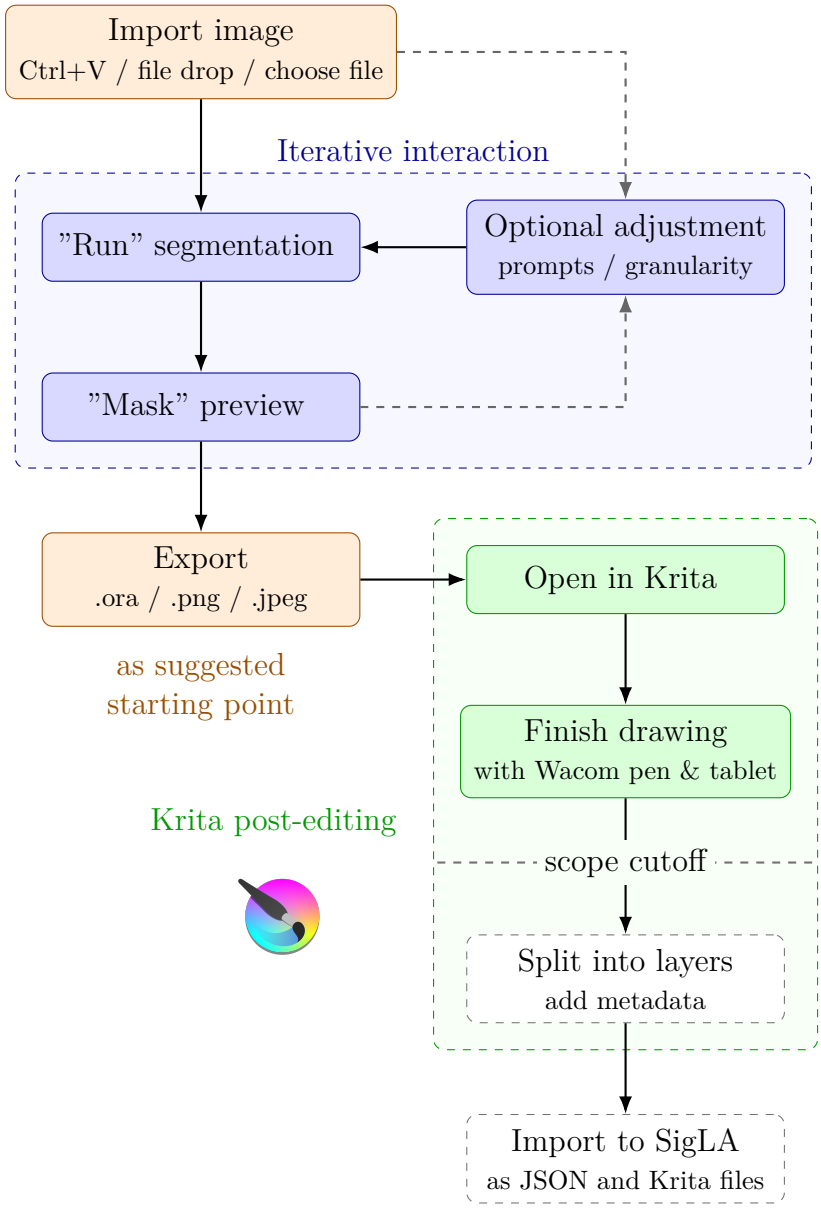


Figure 3.12: User-facing workflow of the developed segmentation tool (in orange, and blue), its iterative interaction (in blue), and its relation to the existing Krita-based annotation workflow (in green). The *scope cutoff* marks the upper limit of the workflow that the tool was designed to partially automate.

3.3.2 The adaptation (export to Krita)

When the segmentation of a document within the tool was complete, it was designed to allow for export to JPEG, PNG, or OpenRaster (`.ora`) format. The latter is an open file format that like Krita (`.kra`) files consists of a ZIP file with an XML document describing the layer structure, together with a PNG or SVG image for each raster or vector layer, respectively. This is simpler in structure and easier to read and write for external programs than the `.kra` file format, since it was specifically designed as an open exchange format between graphics editors. Krita files, on the other hand, are more complex in structure, including proprietary elements, and not designed for external programmatic access, as they are meant to be used within Krita only.

Additionally, OpenRaster (`.ora`) format is supported by a few image editing programs, including Krita, which makes it ideal for the exported file to be directly opened there for further manual editing and annotation. And as Krita was already part of Ester's workflow, it was only natural to make the exported starting point suggestions from the tools directly compatible with it, to make the transition from tool to manual post-editing as seamless as possible. This file format also made it possible for the tool export to include the original image as a layer, and the generated mask as another, so that the former could be used as a guiding reference for post-editing of the latter.

3.3.3 Experimental design for RQ3

To evaluate the workflow efficiency of the two developed tools (see Section 3.2), the time it took Ester Salgarella to do the manual annotation workflow in Krita was compared in a case study of 3 documents to the time taken when doing the complete workflow of each of these tools. The workflow efficiency metric was defined as the sum of tool time (image import to mask export) and drawing time in Krita – which for the tools was defined by post-editing time (polishing the tool-suggested mask draft until completion).

Thus, this workflow efficiency case study was built upon the tool-related data (of tool time and mask exports) obtained from the in-tool case study (see Sections 3.2 and 3.2.3). Hence, both studies suffered from

the same time-constraint-caused limitations in terms of sample size and statistical power, making them exploratory and case-study-based rather than statistically powered.

The drawings in Krita were made using the displayless pressure-sensitive Wacom drawing pad with a pen input used in Ester's original workflow (see Figure 3.12). The case study was not done in-person, but consisted of Ester Salgarella remotely post-editing the tool exports from the interaction efficiency study in Krita, recording the time taken on her phone. For the manual workflow, she was then asked to do the same drawings, but from scratch without any tool-suggested mask draft. And as a proof of completion, Ester was asked to send the finished `.ora` files back again.

Even with the originally imported raw tablet images included as layers in the exported `.ora` files, Ester expressed a lack of guiding reference to know when the drawing was complete. Therefore, the corresponding GORILA corpus expert drawing for each document was manually imported, aligned with the other two layers, and included as an additional layer in the exported `.ora` file before letting Ester start the post-editing. For the manual workflow, all of these layers except the tool-suggested draft were also included, to make the comparison as fair as possible.

CHAPTER 4

Results and Discussion

4.1 Non-interactive segmentation performance

After running the non-interactive evaluation of the models' segmentation quality on the test set of 27 raw tablet images, it was clear that SAM did not perform better than the classical baselines (see Section 4.1.3). It was also found that the tested ICL variants of SAM were beaten (see Section 4.1.2) by the best zero-shot SAM implementation, which through ablation was determined to be the original SAM with classical threshold-derived point prompts, without any filter (see Section 4.1.1).

The following subsections go in-depth with each of these three subresults, and what they mean for the research question at hand:

4.1.1 Ablation of zero-shot SAM

The results of the ablation study of the in-total 12 different permutations of SAM (see Figure 4.1, and Table 4.1) showed that the best performing variant was the original SAM with image encoder Vit-H, without any filter, guided by classical threshold-derived automatic point prompts. This was closely followed by its bilateral pre-filtered variant, and its MobileSAM variant (which performed worse due to its smaller image encoder TinyViT).

Surprisingly so, the SAM2 variants performed worse than the original SAM, both when and when not prompted. The POI-based prompting strategy demonstratively did not work on SAM2, as shown by the SAM2 variants with this strategy performing worse than when unprompted. However, for the other vanilla (non-Hiera) ViT-based SAM variants, the

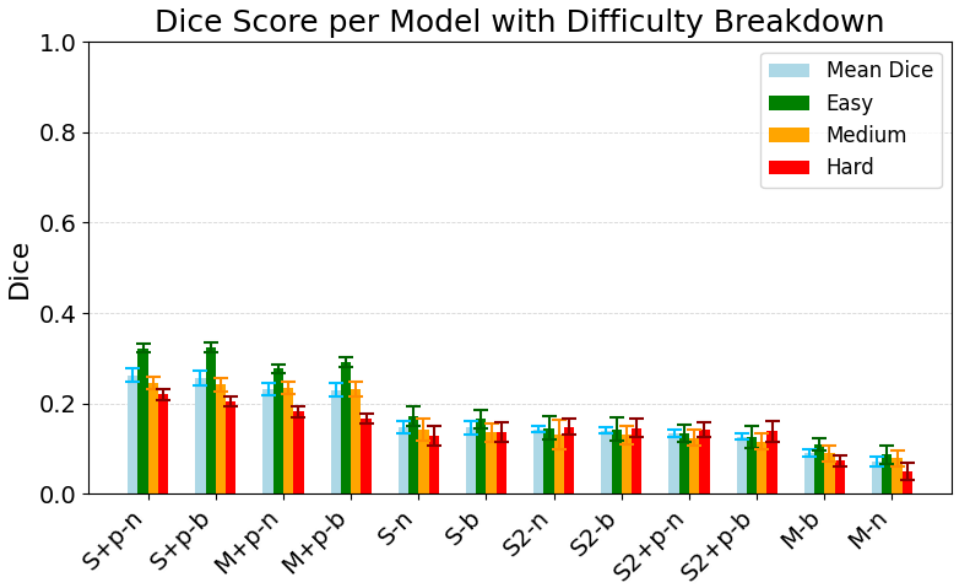


Figure 4.1: Dice Score performance per SAM variant. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance. The ablations evaluated were MobileSAM (M), SAM (S), and SAM2 (S2), with (+p) or without automatic point prompts, and with (b) or without (n) a bilateral pre-filter.

Table 4.1: Average Dice scores of the in total 12 tested permutations of MobileSAM (M), SAM (S), and SAM2 (S2), with and without automatic classical threshold-derived point prompts, and with and without bilateral filter.

	No filter			**Bilateral filter**		
No point prompts	M	S	S2	M	S	S2
	0.071	0.147	0.142	0.091	0.146	0.14
Point prompts	M	S	S2	M	S	S2
	0.231	0.263	0.134	0.229	0.257	0.126

POI-based prompting strategy did work, and was even necessary to obtain any meaningful performance whatsoever. This is not surprising, as zero-shot SAM has never seen this type of images before, since it was exclusively trained on natural images. Thus, it can be stated that when it comes to images of inscribed tablets of Linear A, without any external guidance, the model does not know what to segment,

4.1.2 Comparison to ICL SAM

Neither the one-shot GFSAM nor the few-shot FATESAM performed better (see Figure 4.2) than the best zero-shot SAM implementation (found in the previous Section 4.1.1). GFSAM performed particularly poorly with a mean Dice score of approximately 0.147, obtaining roughly the same poor scores on hard images (0.147), as with both medium (0.143) and easy ones (0.152). FATESAM2D, however, performed relatively much better with a mean Dice score of 0.256, which was comparable to the best zero-shot SAM implementation (with 0.263). It seems that the ICL strategy of GFSAM is not effective within the domain in question, while the ICL strategy of FATESAM2D clearly works, but is still not enough to outperform zero-shot SAM, when the latter has the right automatic prompting strategy.

4.1.2.1 Why GFSAM fails

GFSAM's particularly poor performance was clearly due to the fact that it - for almost all instances - segmented the whole tablet instead of the inscriptions within it (as shown in Figure 4.3). This indicates that the one-shot strategy of GFSAM is generally not effective for this domain of non-natural images. It is very likely that no point ever passed its gating processes, always returning the mask with the highest predicted IoU score instead, which was the whole tablet, as it is always the most salient and clearly separated object in the image. Specifically, GFSAM's overshooting gating process seems to have been the culprit, as it is designed to filter out points that are near the boundary of a mask, which for a thin engraving is always the case.

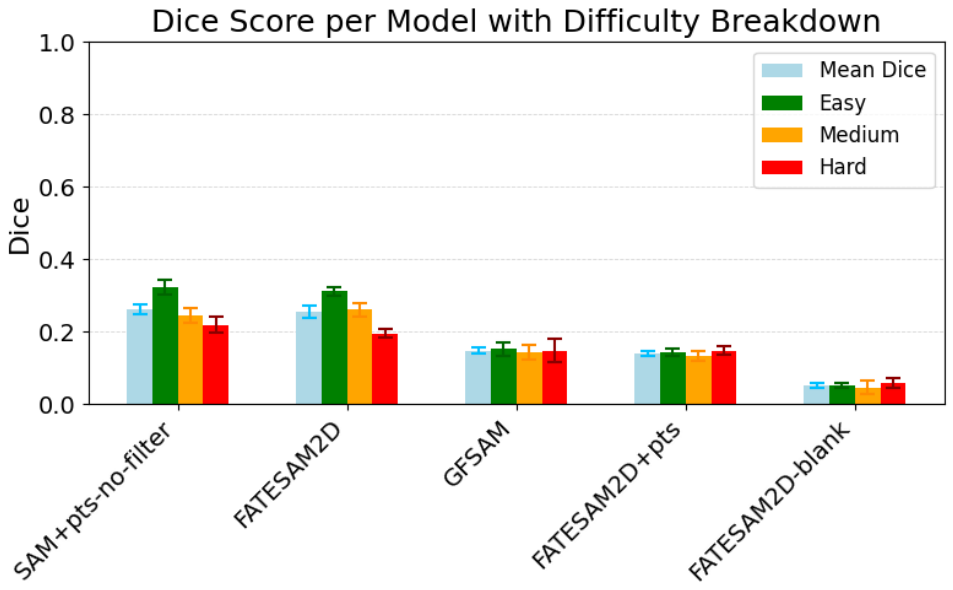


Figure 4.2: Dice scores of the ICL models against the best performing zero-shot SAM implementation. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance. Compared were GFSAM, the 2D-adapted FATESAM2D, its automatic point prompted version FATESAM2D+pts, and FATESAM2D-blank, which was given blank support images.

4.1.2.2 Why FATESAM2D (sort of) works

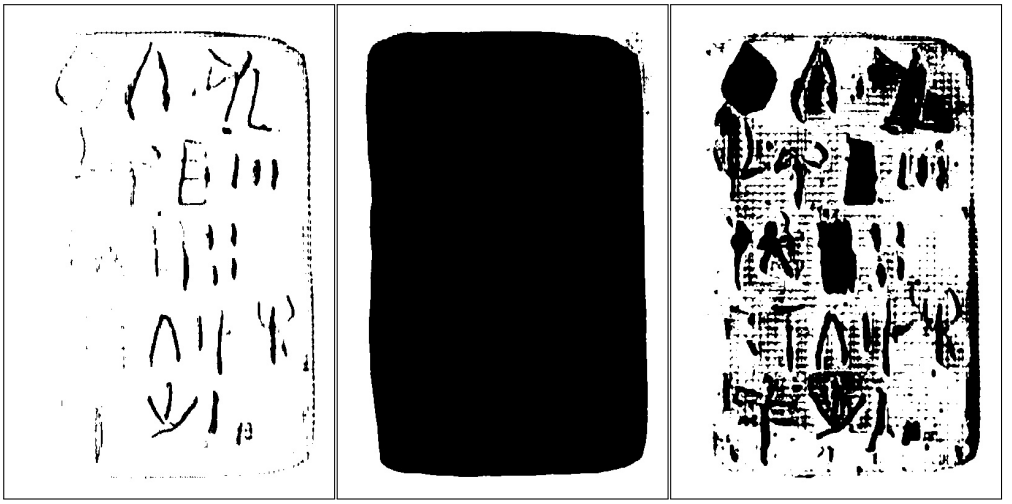
FATESAM2D's comparatively better performance can be attributed to the support images actually providing useful information to the model. The variant with blank support images (FATESAM2D-blank) performing worst of all (see Figure 4.2), is a proof of this, as the inference hence does not work without any meaningful support images. The manipulation of SAM2's memory encoder and memory attention mechanism thus worked reasonably well for few-shot inference given just the raw tablet images.

But still, this additional few-shot context was clearly not enough to outperform the threshold-derived POI-based zero-shot SAM. As a general error, FATESAM2D suffered from undersegmentation rather than oversegmentation (as seen in Figure 4.3). This might have been due

to the big class imbalance between foreground and background in the support images, with background dominating compared to the thin and small engravings.

Furthermore, the POI-based prompting strategy did not work on FATESAM2D, as shown in Figure 4.2 by the variant using this strategy performing worse than when unprompted. This fits with the observation from the previous Section 4.1.1 that this prompting strategy does not work on SAM2, which is where FATESAM2D gets its architecture from.

((a)) Raw Image **((b))** Ground Truth



((c)) FATESAM2D 0.37 **((d))** GFSAM 0.13 **((e))** S+p-n 0.26

Figure 4.3: Visual comparison of output masks per ICL model and best SAM variant from Section 4.1.1 based on the segmentation of PH2 (easy), shown with Dice scores next to the raw image and ground truth. This is one of the instances where FATESAM2D performed better than the best zero-shot SAM implementation, but due to its tendency of undersegmentation, this was generally not the case.

4.1.3 Comparison to classical models

As mentioned already, SAM did not perform better than the classical baselines (see Figure 4.4 for a plot of the results, and Figure 4.5 for a visual comparison). With a mean Dice score of around 0.263, the best performing SAM variant performed better than Otsu (0.224) and CannyFill (0.183), but clearly underperformed compared to the best performing classical baseline, Gaussian (0.327).

In fact, it *significantly* underperformed compared to Gaussian, as is shown statistically in the next Section 4.1.4.

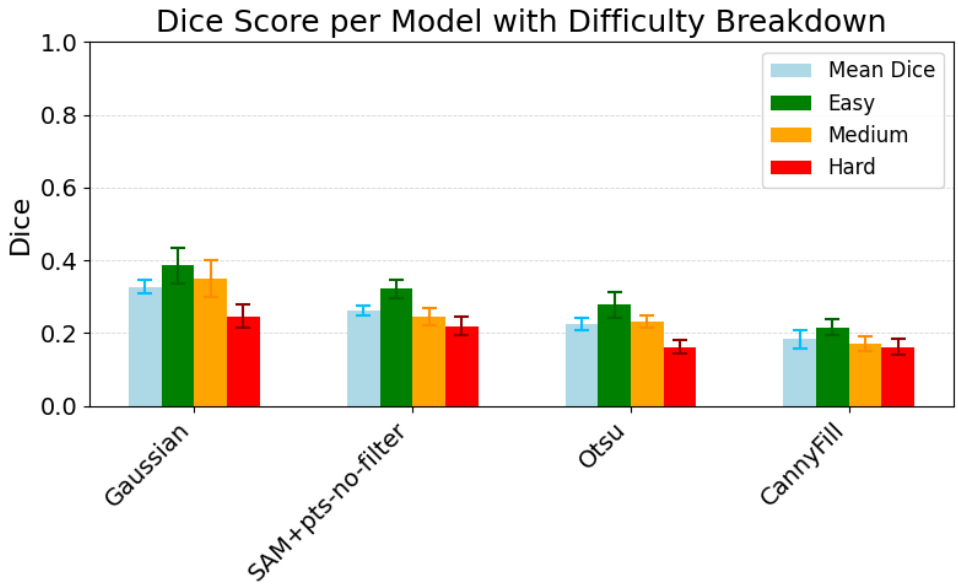
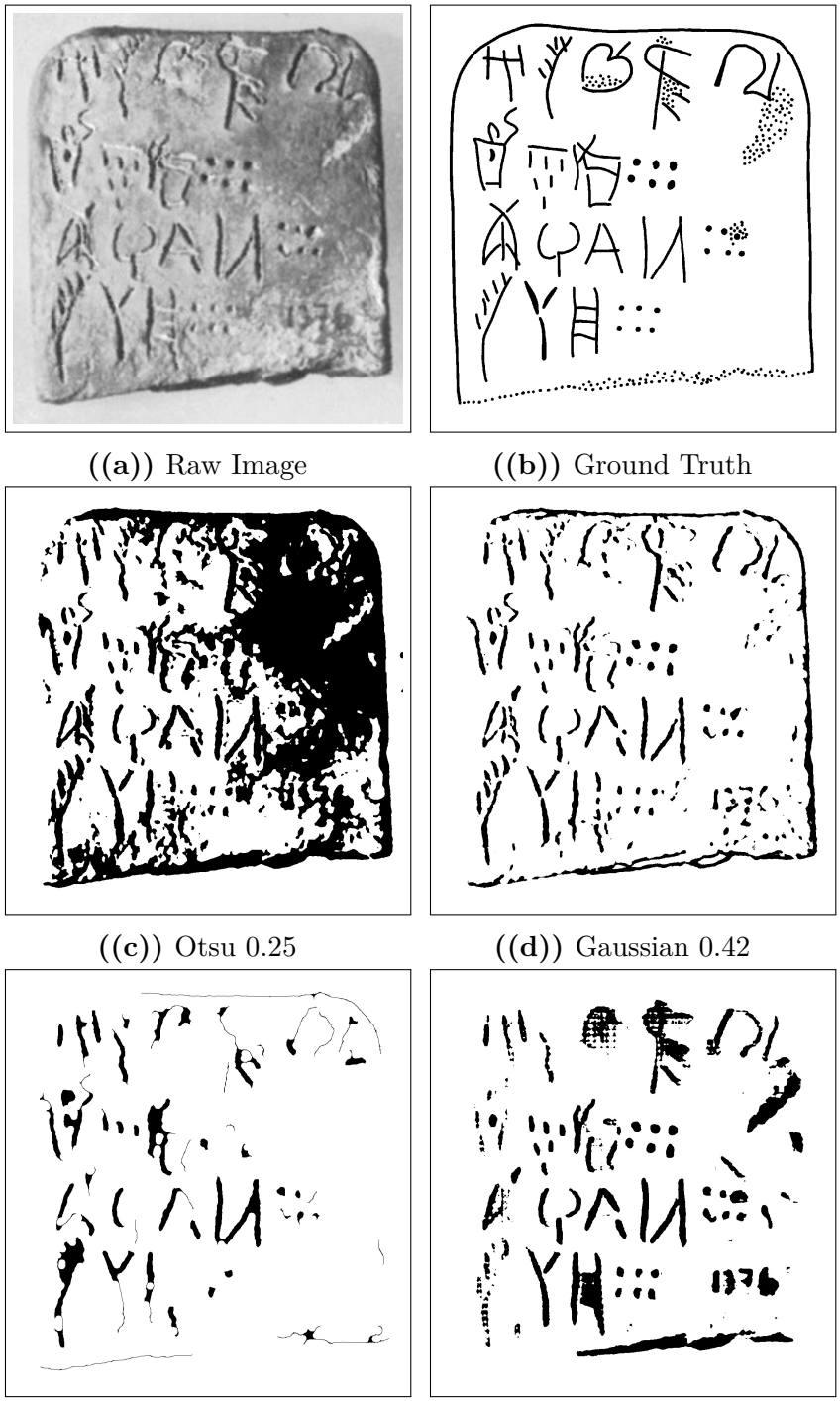


Figure 4.4: Dice scores of the classical baselines compared to the best SAM variant found in Section 4.1.2 and Section 4.1.1. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance.



((**a**)) Raw Image ((**b**)) Ground Truth

((**c**)) Otsu 0.25 ((**d**)) Gaussian 0.42

((**e**)) CannyFill 0.32 ((**f**)) S+p-n 0.38

Figure 4.5: Visual comparison of output masks per classical model and best SAM variant from segmentation of PH2 (easy), shown with Dice scores next to the raw image and ground truth.

4.1.4 Statistical significance

As it was clear from the results, the best tested variant of SAM did not perform better than the best performing classical baseline. To determine whether it performed *significantly worse*, a paired t-test was conducted, as the paired differences in Dice scores between Gaussian and SAM+pts-no-filter were approximately normally distributed (see next Section 4.1.4.1). What follows is the documentation of that normality test, the paired t-test, and the results of both.

4.1.4.1 Normality of paired differences

As the QQplot in Figure 4.6 shows, the paired differences in Dice scores between Gaussian and SAM+pts-no-filter appeared to be approximately normally distributed. Furthermore, the Shapiro-Wilk test for normality of these paired differences yielded a p-value of $0.056 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected. Thus, there was no need for a parametric test like Wilcoxon, as the normality assumption for the paired t-test was satisfied, and its results could be considered valid.

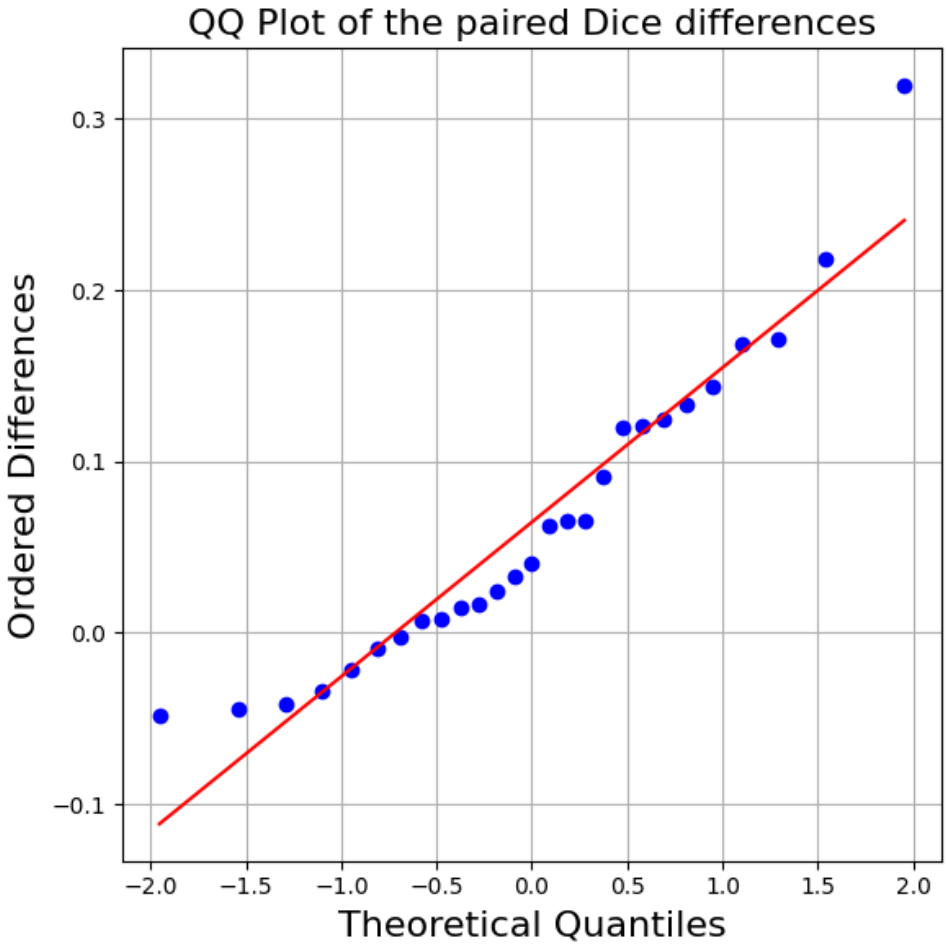


Figure 4.6: QQplot of the paired differences between Gaussian and SAM+pts-no-filter. They appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test on the normality of these paired differences yielded a p-value of $0.056 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected.

4.1.4.2 The paired t-test

A one-tailed paired t-test was conducted on the paired differences d_i in Dice scores obtained for each of the 27 test images across Gaussian and SAM+pts-no-filter (with $d_i = Dice_{Gaussian,i} - Dice_{SAM+pts-no-filter,i}$) under the following hypotheses:

- H_0: There is no difference in mean Dice score between the two models (or SAM has a higher mean).

$$\mu_d \leq 0$$

- H_1: Gaussian has a higher mean Dice score than SAM+pts-no-filter.

$$\mu_d > 0$$

Performing the paired t-test yielded a test statistic of $t \approx 3.737$ and a p-value of $P(T > t) \approx 4.618 \cdot 10^{-4}$. Thus, as $4.618 \cdot 10^{-4} < \alpha = 0.05$, the null hypothesis could be rejected, and it could be concluded that Gaussian significantly outperformed SAM+pts-no-filter in terms of Dice score. In other words, the best tested variant of SAM clearly performed *significantly worse* in comparison to the best tested classical segmentation method for non-interactive segmentation of Linear A tablets.

4.2 Tool-side segmentation performance

The first case study of 3 documents showed that the interactive version of SAM2 with box prompts and point prompts not only took more time to use than the classical baseline (due to the burden of interaction), but also did not perform better in terms of segmentation quality, even with the help of human input (see Figure 4.7).

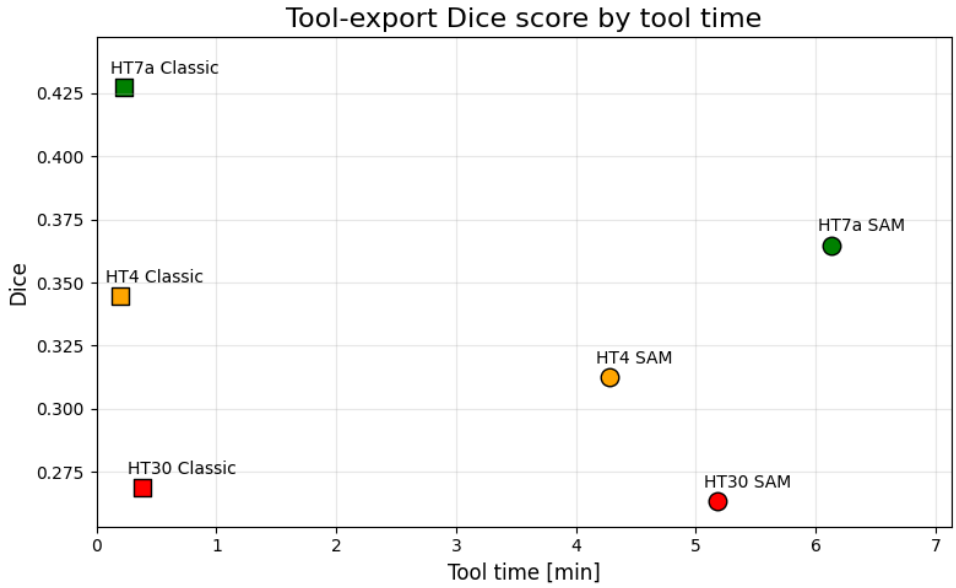


Figure 4.7: Dice scores of the tool-exported masks as a function of tool time, comparing the SAM-based tool to the classical baseline. The three documents tested were HT7a (easy), HT4 (medium), and HT30 (hard). SAM-based tool seemingly both took more time to use, and yielded lower Dice scores than the classical baseline.

4.2.1 Interaction as a burden

Looking at the interaction patterns of the SAM-based tool, it is clear that the amount of prompts placed per image was quite high, with all instances above 60 prompts per image (see Figure 4.8). Furthermore, only background point prompts were placed, suggesting that the user was compensating only for over-segmentation. The iterative refinement was a significant burden on the user, as it required constant attention and interaction, which is both time-consuming and cognitively demanding. And it is this burden of interaction that seems to have been the reason for the huge disparity in time taken compared to the classical baseline (see Figure 4.7).

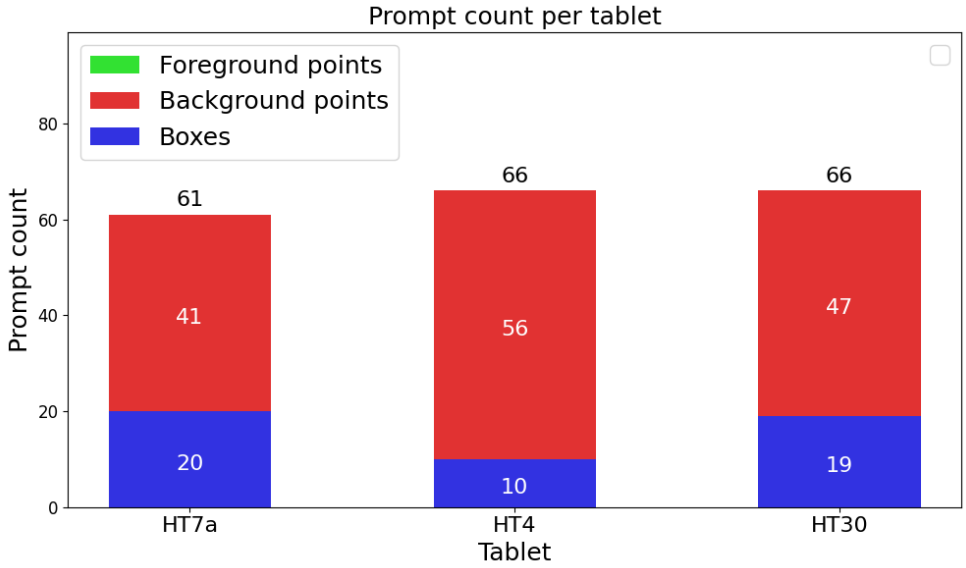


Figure 4.8: The amount of prompts Ester placed per image using the interactive SAM-based tool. No foregound point prompts were placed, suggesting that the user was compensating only for over-segmentation.

4.2.2 Model execution time was not the bottleneck

A significant portion of the time spent using the interactive version of SAM2 was on waiting for the model to execute through Modal (see Figure 4.9). This was a burden on the user, too, as it disrupted the workflow and required waiting time after each execution, which could have been frustrating and have lead to a loss of focus.

But clearly, this was not the bottleneck, as the time spent on interaction was comparatively much higher, taking up more than 80% of tool time. The time spent waiting on SAM execution was furthermore evidently below 5 seconds for more than 50% of executions.

Figure 4.9: Total tool time for all 3 documents spent on model execution per method (left), and the empirical distribution of SAM Modal execution time (right). For SAM, model execution through Modal (via `set_image` and `decode_mask`) took up less than 20% of tool time, while durations were below 5 seconds for more than 50% of all executions.

4.3 Workflow impact

The second case study of the 3 documents showed that none of the two tools were able to save time in the workflow, as the total workflow time spent was higher than the time spent on manual annotation in Krita (see Figure 4.10). Even just based on the time spent on post-editing alone, the tools did not save time compared to just drawing manually from scratch. In fact, the total time spent doing the classical baseline workflow was more than twice the time spent doing the manual one, while the time spent doing the SAM-based workflow was more than thrice (and almost four times) the manual workflow time.

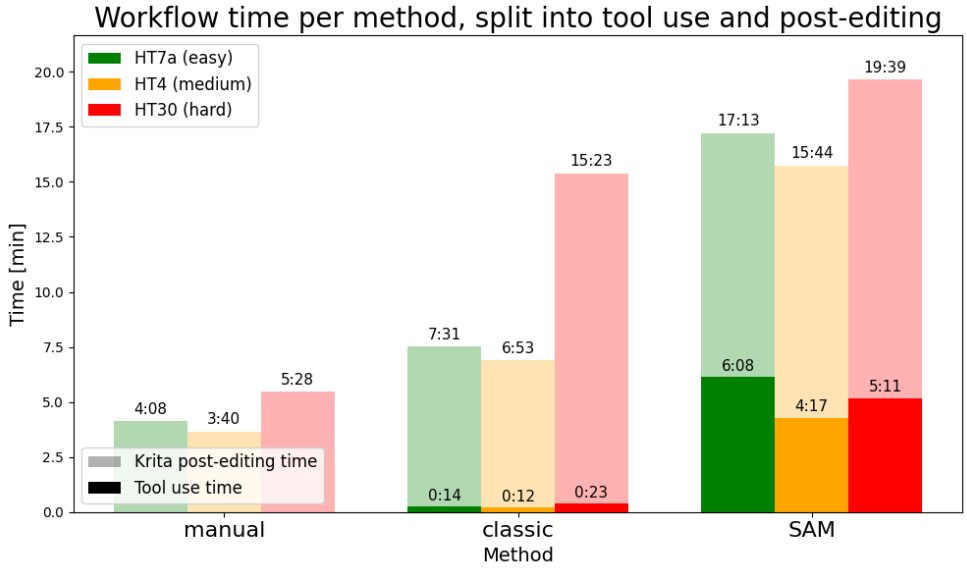


Figure 4.10: Ester's workflow time spent per method and tablet, split up into phases of tool use (opaque colors) and post-editing in Krita (semi-transparent colors). The total time spent doing the classical baseline workflow was more than twice the time spent doing the manual one, while the time spent doing the SAM-based workflow was more than thrice (and almost four times) the manual workflow time. Clearly, it took less time to just draw the masks manually from scratch in Krita, than to use either of the two tools and post-edit the results.

4.3.1 Higher Dice seemingly implies time-saving

When looking at the post-editing times taken in Krita as a function of the tool-export Dice scores, it seems that higher Dice scores do imply time-saving in post-editing (see Figure 4.11). This is not surprising, as higher Dice scores indicate closer resemblance to ground truth, which should require less time to refine in Krita. Thus, the reason that the tools did not save any time, was in all likelihood due to the low resemblance of the exports to ground truth, as indicated by their low Dice scores. But apparently, post-editing time also seems to depend on the size of the document. For both tools, the medium-difficulty document HT4 took about as much or less time to post-edit than the easy-difficulty document HT7a. This can be explained by their size-difference, which also shows in Figure 4.8, as HT7a had 20 box prompts placed (indicating 20 separate engravings), while HT4 had only 10 placed (indicating a smaller tablet).

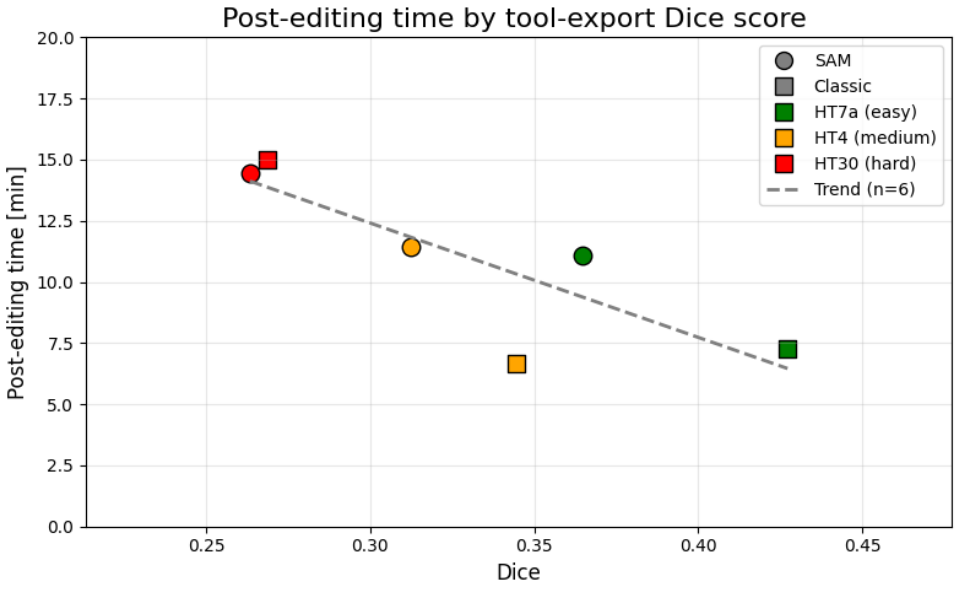


Figure 4.11: Post-editing time taken in Krita as a function of tool-export Dice scores. The trend line cannot be considered statistically significant (as $n = 6$), but points in the direction of a negative correlation.

4.4 Consolidated discussion

The overall results suggest that SAM in the forms tested is not suitable for segmentation of Linear A tablets. For each RQ, SAM failed compared to the baselines - even when guided by human interaction. Not only did it not live up to its supposedly SOTA superiority in terms of mere segmentation quality. It also took more time to use, and did not save any time in the workflow whatsoever.

Nevertheless, it is important to point out that the results of the two case studies (in contrast to the non-interactive evaluation) - due to the small sample size of 3 documents, and the fact that only one user (Dr. Ester Salgarella) was involved - are not statistically generalizable. Thus, the results of these case studies should be interpreted as indicative of the *potential* impact of the tools on the workflow, rather than conclusive evidence of their actual impact. Still, none of the data obtained showed potential for time-saving. And none showed potential for SAM outperforming the best classical baseline.

When taking the domain and data quality into account, this underperformance is not too surprising. The raw document images used for inference are screenshots of grayscale photographs embedded in scanned print editions of GORILA. They therefore provide only a limited two-dimensional representation, through which crucial information for distinguishing between damage and engraving, such as apparent depth and texture, may have been lost. During the second case study, Dr. Salgarella gave the impression that even she herself could not clearly distinguish tablet damage from inscriptions based on the raw images alone. If even an expert cannot do that, it is not surprising that a segmentation model cannot do it either.

But what these results suggest, is not only that the available data is poor. As SAM performed worse than the strongest classical baseline on the same data, it also tells us that SAM's generalization capabilities are not sufficient enough within this domain. For SAM to be useful, not only is there a need for a more suitable data source. To be able to adapt it to the domain at all, there is seemingly also a need for a sufficiently sized dataset. Although tuning as a proof of concept might not require more than a few hundred datapoints (as seen in the study of SAM on Mayan

hieroglyphs, Shivam et al. 2024). To be able to create a model that can be used in practice, a dataset of at least a few thousand annotated images would likely be necessary. Creating a hypothetical dataset of such size might require collecting inscriptions other than just Linear A - such as the other closely related Aegean scripts like Linear B, and maybe even epigraphical inscriptions in general - laying the groundwork for the creation of an epigraphically tuned SAM.

Until such a model is developed, however, the results seem to suggest that classical deterministic methods are better suited for the task. But most importantly, they suggest that manual annotation in Krita so far remains the best option for producing digitized drawings of Linear A documents. Nevertheless, that is not to say that there is no potential for automation in the workflow. Rather, it means that the tested approaches are not sufficient enough to be helpful for it, and that alternative approaches should be explored in future work.

In particular, approaching the drawing step of Ester's annotation workflow as a segmentation task on raw document images does not seem to be very useful. And considering the quality of these images, it probably never will be - even when tuning on a big hypothetical dataset like the one mentioned. However, if the INSCRIBE (Ravanelli, Lastilla, and Ferrara 2022) project were to resume and succeed in producing high-quality 3D scans for the entire corpus, a 3D segmentation approach might very well prove useful. The process of producing such a complete set of scans, however, would arguably take longer than Ester merely manually producing digital drawings for the entire corpus.

Setting IPR issues aside, the best information available is clearly the expert drawings from the GORILA corpus. They exist for all published documents and are so close to the ground truth that they could arguably be seen as ground truth themselves. In theory, it is entirely doable to cut drawing time through little else than binarization, rescaling and smoothing of those drawings, giving Ester the most qualified starting points possible, which she can then adapt to her needs. Doing so, however, was not within the scope of this thesis, as it was focused on the general process of creating drawings from scratch through segmentation. But in Ester's case specifically, it represents a promising workaround and pivot for future work, as such an approach does not depend on any segmentation models nor any cumbersome data bottlenecks whatsoever.

CHAPTER 5

Conclusion

The aim of this thesis was to investigate whether the drawing step of the annotation workflow of digitizing Linear A inscriptions can be partially automated by approaching the problem as a segmentation task on raw document images from GORILA, using the Segment Anything Model (SAM). In short, under the tested conditions, the results indicate that SAM was not practically useful for accelerating Salgarella's workflow of annotating Linear A tablets. This is illustrated by the following summaries of findings per research question.

5.1 RQ1: Segmentation quality

The first research question concerned the non-interactive segmentation quality of SAM within the domain of Linear A document images compared to classical image processing baselines. Measured with Dice scores on a test set of 27 images, none of the tested SAM variants outperformed the strongest classical baseline, Gaussian thresholding. On the contrary, the strongest SAM variant performed *significantly worse*, despite the inclusion of ICL implementations and strategic POI prompt placement.

5.2 RQ2: Interaction efficiency

The second research question had to do with the efficiency of using SAM as an interactive segmentation tool compared to a classical image processing baseline. This was measured in a case study of 3 documents from the test set with Ester Salgarella in terms of the time spent in the tool and segmentation quality of the exports. Despite the theoretical ben-

efit of human interaction, the SAM2-based tool seemed beaten by the Gaussian thresholding one both in terms of time spent and segmentation quality.

5.3 RQ3: Workflow efficiency

The third research question considered the workflow-related efficiency of using the SAM-based tool compared to classical image processing and the baseline of manual annotation. Measured in a case study of 3 documents with Ester Salgarella, the time spent to complete the drawings manually in Krita was compared to being assisted by the mask exports of the previous case study as suggested starting points. None of the tools showed any potential of saving time in the workflow, both when considering the entire workflow (including the time spent in tool), and when considering only the post-editing phase. In fact, the tools even seemed to increase work time (SAM more so than the classical one), as their output masks required too much correction.

5.4 Overall conclusion

To reiterate, SAM was not deemed practically useful for accelerating Salgarella's workflow of annotating Linear A tablets. This was mostly due to the limitations of the raw document images from GORILA used for inference, which was low both in size and quality, consisting of screenshots of grayscale photographs embedded in scanned print editions of GORILA. But SAM also seemed to struggle with the domain of Linear A documents in general, as neither zero-shot nor few-shot implementations outperformed the classical baselines on segmentation quality. Even when guided by human interaction, the performance of SAM did not reach the level of the classical baselines. This can be attributed to the fact that SAM was pre-trained on natural images and therefore lacks the domain-specific knowledge to perform well on the task of segmenting Linear A tablets, which are characterized by several thin, fragmented signs as opposed to the more distinct "rounded" objects found in natural images.

Thus, the results suggest that SAM needs tuning to perform well on this task. Although the results were clearly negative, they provide valuable insights into the shortcomings of SAM within the domain of GORILA raw document images, and suggest directions for future research and development to achieve a more automated workflow.

5.5 Future Work

5.5.1 Alternative data sources

Since the raw tablet based ML system developed did not perform well enough to be deemed helpful for the workflow, another approach seems necessary in order to achieve a more automated workflow. The bottleneck of the raw tablet based approach seems to be the quality of the input data, as the raw images are of low quality and contain a lot of noise, which is not only hard for the model to process, but also even for the researcher to differentiate the signs from mere tablet damage. The suggestions generated should therefore instead be inferred from the corpus drawings, which are of higher quality, carry more strucured information than the raw tablet images, and are what the researcher already primarily uses as reference in the annotation process.

A complementary, higher-quality data source would be the 3D scans of the INSCRIBE project (Ravanelli, Lastilla, and Ferrara 2022), on which a 3D segmentation approach might prove useful. This would, however, require the project to resume and to scan the remainder of the corpus, a process that would arguably take longer than the manual production of digital drawings for the entire corpus.

5.5.2 Methodological extensions

Beyond a change of data source, the results suggest a need for a generalized and computationally efficient SAM implementation, tuned to the greater domain of inscriptions, as the default model was not sufficient to demonstrate its potential capabilities within this domain. For such tuning to be possible, a sufficiently sized one-class-labeled dataset of gen-

eral inscription segmentations would be required. As a proof of concept, domain tuning might not demand more than a few hundred annotated examples, as demonstrated for Mayan hieroglyphs (Shivam et al. 2024). A model usable in practice, however, would likely require a dataset of at least a few thousand annotated images. Assembling a dataset of that size for Linear A alone is unrealistic given its small corpus, so it would likely be necessary to pool inscriptions from the closely related Aegean scripts such as Linear B, and possibly from epigraphic inscriptions more broadly, laying the groundwork for an epigraphically tuned SAM.

5.5.3 Further automation opportunities

Until such a model and dataset exist, classical deterministic methods appear better suited to the task than SAM, and manual annotation in Krita remains the most efficient option for producing digitized drawings of Linear A documents. This does not imply that the workflow holds no potential for automation, but rather that the approaches tested here are insufficient and that alternative ones should be explored. The most immediate opportunity lies in reframing the drawing step away from segmentation on raw document images, and instead deriving suggested starting points from the existing expert corpus drawings, which are already close to ground truth and could be processed through little more than binarization, rescaling, and smoothing. Looking further ahead, the metadata-labeling step of the annotation process, which this thesis did not address, presents an additional target for automation. And should an epigraphically tuned model eventually be realized, the same workflow could in principle be extended beyond Linear A to Linear B and other under-digitized epigraphic scripts, contributing to the broader field of digital epigraphy.

Bibliography

Bergstra, James, Remi Bardenet, and Balázs Kégl (December 2011). "Algorithms for Hyper-Parameter Optimization." In.

Bergstra, James, Daniel Yamins, and David D. Cox (2013). "Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures." In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Volume 28. Proceedings of Machine Learning Research 1. PMLR, pages 115–123. URL: https://proceedings.mlr.press/v28/bergstra13.pdf.

Carion, Nicolas et al. (2025). *SAM 3: Segment Anything with Concepts*. arXiv: 2511.16719 [cs.CV]. URL: https://arxiv.org/abs/2511.16719.

Castellan, Simon (2026). *SIGIL. Store and Explore Databases of Ancient Inscriptions in Linear A*. URL: https://gitlab.inria.fr/sicastel/sigil (visited on June 5, 2026).

Chen, Danlu et al. (2026). *LogogramNLP: Comparing Visual and Textual Representations of Ancient Logographic Writing Systems for NLP*. arXiv: 2408.04628 [cs.CL]. URL: https://arxiv.org/abs/2408.04628.

Christoffersen, Frederik W. L. (2026). *Source code for this Thesis*. Source code repository. URL: https://github.com/FrederikWLC/linA-scribe.

Consiglio Nazionale delle Ricerche (2025). *A Major Addition to the Linear A Corpus*. URL: https://www.cnr.it/en/news/13529/a-major-addition-to-the-linear-a-corpus (visited on February 4, 2026).

Del Freo, Maurizio and Julien Zurbach (2025). *Recueil des inscriptions en linéaire A. Supplément 1 (RILA-S1)*. Athens: École française d'Athènes.

Dosovitskiy, Alexey et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv: 2010.11929 [cs.CV]. URL: https://arxiv.org/abs/2010.11929.

ERC INSCRIBE Project (2026). *INSCRIBE 3D Interactive Web Viewer*. University of Bologna. URL: https://www.inscribercproject.com/Linear_A.php (visited on June 2, 2026).

Godart, Louis and Jean-Pierre Olivier, editors (1976-1985). *Études crétoises. Recueil des inscriptions en linéaire A (GORILA)*. Five-volume corpus of Linear A inscriptions (GORILA I–V). Athènes. URL: https://cefael.efa.gr/result.php?site_id=1&serie_id=EtCret.

Gonzalez, Rafael C. and Richard E. Woods (2018). *Digital Image Processing*. 4th edition. New York: Pearson. ISBN: 978-0133356724. URL: https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf.

Hamplová, Adéla et al. (2024). "Instance Segmentation of Characters Recognized in Palmyrene Aramaic Inscriptions." In: *Computer Modeling in Engineering & Sciences* 140.3, pages 2869–2889. ISSN: 1526-1506. DOI: 10.32604/cmes.2024.050791. URL: http://www.techscience.com/CMES/v140n3/57249.

Han, Dongsheng et al. (2023). *Segment Anything Model (SAM) Meets Glass: Mirror and Transparent Objects Cannot Be Easily Detected*. arXiv: 2305.00278 [cs.CV]. URL: https://arxiv.org/abs/2305.00278.

He, Xingxin et al. (2025a). *FATE-SAM: Few-Shot Adaptation of Training-Free Foundation Model for 3D Medical Image Segmentation*. GitHub repository. URL: https://github.com/I3Tlab/FATE-SAM (visited on May 23, 2026).

— (2025b). *Few-Shot Adaptation of Training-Free Foundation Model for 3D Medical Image Segmentation*. arXiv: 2501.09138 [cs.CV]. URL: https://arxiv.org/abs/2501.09138.

Kirillov, Alexander et al. (2023). "Segment Anything." In: *arXiv preprint arXiv:2304.02643*. URL: https://arxiv.org/pdf/2304.02643.

Ma, Jun et al. (January 2024). "Segment anything in medical images." In: *Nature Communications* 15.1. ISSN: 2041-1723. DOI: 10.1038/s41467-024-44824-z. URL: http://dx.doi.org/10.1038/s41467-024-44824-z.

Meta AI Research (2024). *Segment Anything Model 2*. GitHub repository. URL: https://github.com/facebookresearch/sam2 (visited on May 23, 2026).

Modal Labs (2026). *Modal Documentation*. Documentation for the Modal cloud execution platform. URL: https://modal.com/docs/guide (visited on May 23, 2026).

Mohammadi, Mobin, Kaveh Mollazade, and Nasser Behroozi-Khazaei (2025). "Under- and over-segmentation: New metrics for image segmentation accuracy measurement." In: *Array* 28, page 100624. ISSN: 2590-0056. DOI: https://doi.org/10.1016/j.array.2025.100624. URL: https://www.sciencedirect.com/science/article/pii/S2590005625002516.

Oliveira, Saïd, Benoît Seguin, and Frédéric Kaplan (2018). "dhSegment: A generic deep-learning approach for document segmentation." In: *arXiv preprint arXiv:1804.10371*. URL: https://arxiv.org/abs/1804.10371.

OpenCV (2026a). *cv::AdaptiveThresholdTypes — OpenCV Documentation*. OpenCV. URL: https://docs.opencv.org/4.x/d7/d1b/group__imgproc__misc.html#gaa42a3e6ef26247da787bf34030ed772c (visited on March 7, 2026).

— (2026b). *Image Filtering: Bilateral Filter*. URL: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#ga9d7064d478c95d60003cf8 (visited on March 23, 2026).

Oquab, Maxime et al. (2024). *DINOv2: Learning Robust Visual Features without Supervision*. arXiv: 2304.07193 [cs.CV]. URL: https://arxiv.org/abs/2304.07193.

Otsu, Nobuyuki (1979). "A Threshold Selection Method from Gray-Level Histograms." In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1, pages 62–66. DOI: 10.1109/TSMC.1979.4310076. URL: https://engineering.purdue.edu/kak/computervision/ECE661.08/OTSU_paper.pdf.

Pan, Cong (2025). "Real-Time Hardware Architecture Design for An Innovative Bilateral Filtering Algorithm." In: *Proceedings of the 2025 5th International Conference on Automation Control, Algorithm and Intelligent Bionics (ACAIB '25)*. New York, NY, USA: ACM, pages 102–107. ISBN: 9798400714313. DOI: 10.1145/3760269.3760285. URL:

https://dl.acm.org/doi/10.1145/3760269.3760285 (visited on March 23, 2026).

Price, Stephen, Elke Rundensteiner, and Danielle L. Cote (2025). "Mastering SAM Prompts: A Large-Scale Empirical Study in Segmentation Refinement for Scientific Imaging." In: *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: https://openreview.net/forum?id=cWcTQMpqv6.

Rainio, Oona, Jarmo Teuho, and Riku Klén (2024). "Evaluation metrics and statistical tests for machine learning." In: *Scientific Reports* 14, page 6086. DOI: 10.1038/s41598-024-56706-x. URL: https://doi.org/10.1038/s41598-024-56706-x.

Ravanelli, Roberta, Lorenzo Lastilla, and Silvia Ferrara (2022). "A high-resolution photogrammetric workflow based on focus stacking for the 3D modeling of small Aegean inscriptions." In: *Journal of Cultural Heritage* 54, pages 130–145. ISSN: 1296-2074. DOI: https://doi.org/10.1016/j.culher.2022.01.009. URL: https://www.sciencedirect.com/science/article/pii/S1296207422000097.

Ravi, Nikhila et al. (2024). *SAM 2: Segment Anything in Images and Videos*. arXiv: 2408.00714 [cs.CV]. URL: https://arxiv.org/abs/2408.00714.

Render (2026). *Render: The Cloud for Builders*. URL: https://render.com/ (visited on June 6, 2026).

Rosebrock, Adrian (2015). *Zero-parameter, automatic Canny edge detection with Python and OpenCV*. Accessed: 2026-04-06. URL: https://pyimagesearch.com/2015/04/06/zero-parameter-automatic-canny-edge-detection-with-python-and-opencv/.

Ryali, Chaitanya et al. (2023). *Hiera: A Hierarchical Vision Transformer without the Bells-and-Whistles*. arXiv: 2306.00989 [cs.CV]. URL: https://arxiv.org/abs/2306.00989.

Salgarella, Ester (2025). *Writing in Bronze Age Crete: 'Minoan' Linear A*. Cambridge: Cambridge University Press. URL: https://doi.org/10.1017/9781009520041.

Salgarella, Ester and Simon Castellan (2020). *SigLA: The Signs of Linear A: a Palaeographical Database*. Research Paper / Technical Report. Accessed: 2026-02-04. SigLA. URL: https://sigla.phis.me/paper.pdf.

— (2026a). *About SigLA*. URL: https://sigla.phis.me/about.html (visited on June 5, 2026).
— (2026b). *SigLA Document Search Interface*. URL: https://sigla.phis.me/search-document.html (visited on June 5, 2026).
Shivam, FNU et al. (2024). *Segmentation of Maya hieroglyphs through fine-tuned foundation models*. arXiv: 2405.16426 [cs.CV]. URL: https://arxiv.org/abs/2405.16426.
Tang, Lv, Haoke Xiao, and Bo Li (2023). *Can SAM Segment Anything? When SAM Meets Camouflaged Object Detection*. arXiv: 2304.04709 [cs.CV]. URL: https://arxiv.org/abs/2304.04709.
Tomasi, Carlo and Roberto Manduchi (1998). "Bilateral Filtering for Gray and Color Images." In: *Proceedings of the Sixth International Conference on Computer Vision*. Bombay, India: IEEE, pages 839–846. DOI: 10.1109/ICCV.1998.710815. URL: https://users.soe.ucsc.edu/~manduchi/Papers/ICCV98.pdf (visited on March 23, 2026).
Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.
Wang, Shibin et al. (2025). "OBIFlo-SAM: multi-task semantic recognition and segmentation of oracle bone inscription." In: *npj Heritage Science* 13, page 588. DOI: 10.1038/s40494-025-02157-0. URL: https://www.nature.com/articles/s40494-025-02157-0.
Wu, Kan et al. (2022). "TinyViT: Fast Pretraining Distillation for Small Vision Transformers." In: *arXiv preprint arXiv:2207.10666*. arXiv: 2207.10666 [cs.CV]. URL: https://arxiv.org/abs/2207.10666.
Ye, Maoyuan et al. (2024). *Hi-SAM: Marrying Segment Anything Model for Hierarchical Text Segmentation*. arXiv: 2401.17904 [cs.CV]. URL: https://arxiv.org/abs/2401.17904.
Zhang, Anqi et al. (2024a). *Bridge the Points: Graph-based Few-shot Segment Anything Semantically*. arXiv: 2410.06964 [cs.CV]. URL: https://arxiv.org/abs/2410.06964.
— (2024b). *GF-SAM: Official Code for Graph-based Few-shot Segment Anything Semantically*. URL: https://github.com/ANDYZAQ/GF-SAM (visited on June 6, 2026).
Zhang, Chaoning et al. (2023). "Faster Segment Anything: Towards Lightweight SAM for Mobile Applications." In: *arXiv preprint arXiv:2306.14289*. URL: https://arxiv.org/pdf/2306.14289.

Zhang, Pan, Chao Li, and Yuanhua Sun (2024). "Stone Inscription Image Segmentation Based on Stacked-UNets and GANs." In: *Discover Applied Sciences* 6.10, page 550. DOI: 10.1007/s42452-024-06264-8. URL: https://doi.org/10.1007/s42452-024-06264-8.

Zhao, Xu et al. (2023). "MobileSAMv2: Faster Segment Anything to Everything." In: *arXiv preprint arXiv:2312.09579*. URL: https://arxiv.org/pdf/2312.09579.

APPENDIX A

An Appendix

A.0.1 Examples of hyperparameter configurations

X

1. C: the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.1.2.3),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.1.2.3),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral prefiltering (see Section 3.1.2.3),
4. σ: the spatial and intensity standard deviation for bilateral prefiltering (see Section 3.1.2.3),

Figure A.1: Hyperparameters for Gaussian adaptive thresholding (Gaussian).

X

1. σ: the constant used to determine the thresholds T_l and T_h for the hysteresis thresholding,
2. $d_{closing}$: the diameter of the kernel window for the morphological closing of the edges

Figure A.2: Hyperparameters for Canny edge detection with morphological closing (CannyFill).

X

1. C: the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.1.2.3),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.1.2.3),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral prefiltering (see Section 3.1.2.3),
4. σ: the spatial and intensity standard deviation for bilateral prefiltering (see Section 3.1.2.3),
5. n_{fgd_points}: the number of foreground point prompts to be sampled from the foreground of the first Gaussian thresholded mask,
6. n_{bgd_points}: the number of background point prompts to be sampled from the sure background region,
7. $d_{gap\_erosion}$: the kernel size for erosion used to create a gap between the foreground and background regions.

Figure A.3: Hyperparameters for SAM with automatic point prompts (SAM+pts)

A.1 Interactive tool development

A.1.1 Usability tests

A.1.1.1 Requirement analysis and first usability test

The 5th of May, a requirement analysis and first usability test of a MVP was performed with Ester through a video call. That MVP was composed of two tools: a classical method (Gaussian), and an interactive SAM version (MobileSAM). After trying out the tools, she gave the impression that she could probably use the outputs as draft drawings to be completed with a pen, which might save her some time.

For the classical baseline, the grainy resolution (due to the input image quality) bothered her a bit. The output mask was also noisy at times. And not all lines were always picked up.

For the SAM version, she pointed out that it did not pick up the noisy tablet damage as much as the classical method did. But she was worried that the interaction with SAM would take longer than simply doing strokes manually. The box-prompt workflow was not completely successful during the test. Due to bugs, boxes mostly did not work. and when they did, the segmented forms were not useful.

A.1.1.2 Second (small) usability test

The 8th of May, another usability test was performed on an updated MVP with Ester through a video call. Now, the box prompts worked, and the MobileSAM was replaced with SAM2.1. The segmentation quality per box-prompted sign was noticeably better than previously. Together with Lukas Esterle, the conclusions were to make a slider for the classical model to decrease and increase granularity. It was also suggested that box prompts for SAM could be computationally preselected based on the mask from the classical model, but this was discarded as it was not deemed needed.

A.1.1.3 Third usability and final acceptance test

The 12th of May, a third usability test was conducted in person at AIAS in Århus, including the first part of the usability experiment. As all 45 images could not be drawn all at once, the .ora exports from the tool were saved, and finished at a later time in batches by Ester on her own, for each of them recording the time it took to do so. Due to the huge amount of time it took to process the images with the tool (especially with the SAM version), the experiment was stopped after just 3 images.

$\sqrt{17}$
$\Omega \int \delta e^{i\pi} = -1$
{2.7182818284}ϑφερτυθιοπσδφγηξκλ
χ^2
$\sum$
λ

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk